



Industry Internship
**Electrical Engineering and Information
Science**

Karlsruhe, September 23, 2022

Internship Report

Richard Fabian Willke

Duration: 25.4.2022 - 23.9.2022

Supervisors: Dexter Chew, Raymond Chan

Abstract

This document is a summary of the tasks I worked on during the time of my internship as an electrical engineer in Advantir Innovations, Singapore. The summary is also being used as a documentation for the engineering department of Advantir Innovations. The documentation will be used as a reference for further development and understanding of the presented products.

Contents

1 Company	4
1.1 Department	4
1.2 Task	5
2 Project 1: v4 UI design	6
2.1 Subtask 1: circuit design	7
2.2 Subtask 2: UI programming	9
2.3 Subtask 3: hardware implementation	12
3 Project 2: construction of the auger insertion mechanism	15
3.1 Subtask 1: noise reduction	17
3.2 Subtask 2: auger insertion control	23
3.3 Subtask 3: UI design	27
3.4 Subtask 4: PCB design	29
3.5 Subtask 5: BOM and documentation	36
3.6 Subtask 6: second generation PCB	36
4 Project 3: torque control for the motor of the v3.9.5 machine	39
4.1 cost down	39
5 Project 4: ease of tests and machine usage	41
5.1 cup spinner for testing	41
5.2 video analysis of the fluid flow	43
5.3 mixed-in size-detector	58
6 Project 5: construction of stepper motor controller	68
7 Conclusion	71
8 Future	72
9 List of Abbreviations	73
10 Appendix	78

1 Company

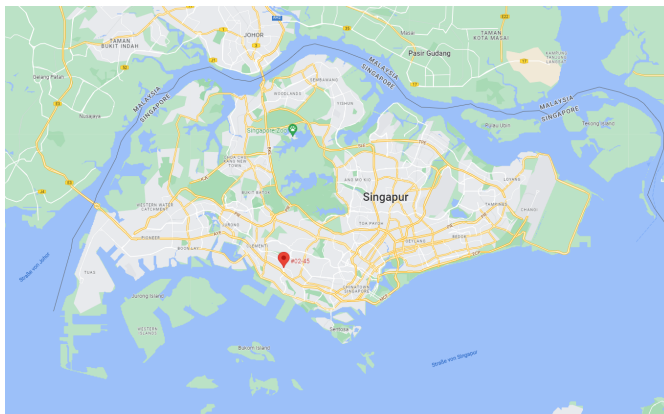
1.1 Department



Figure 1: Swirl.Go logo [2]

Blk 81 Ayer Rajah Crescent,
Singapore 139967
<https://www.swirlgo.com/>

Advantir Innovations is a company, founded and located in Singapore. Their product is the Swirl.GO soft-serve ice cream machine. Conventional soft serve machines are expensive, hard-to-maintain and mostly only serve 1-2 flavours. The Swirl.GO in the other hand has a variety of flavours to choose from, while it is easy for the user to maintain. The countertop-sized machine has zero food contact requires no cleaning.



(a) Location relative to Singapore island



(b) Location relative to station One North

Figure 2: Location of Advantir Innovations [3]

I worked in the company's engineering and development department. The office is located on the second floor, office number 45. I shared the office space with my supervisor Dexter Chew and my engineering coworkers.

The engineering department works on realising solutions and improving already existing concepts of the Swirl.GO models. The main fields are the improvement of the machines' mechanics and mechatronic components, the development of machine oriented algorithms and the improvement of the ice cream formula for each flavour. The tasks, I was involved with, revolved around the development of mechatronical and information scientific solutions for the improvement of existing machines and creation of future models.

1.2 Task

My tasks were located in the development and testing of mechatronic systems and their embedded systems as well as the development of the used testing methods.

The mechatronical tasks were primarily focused on the development of electrical circuits and microcontroller programming. The tests on the machines were focused on the interaction of electrical and mechanical components within the machines and their environment.

The development of testing methods was mostly based on programming with computer vision. This part belongs into the information scientific part of my tasks. The testing methods were used to gather specific data and process it to usable results.

2 Project 1: v4 UI design

The goal of this task is the design of an electrical circuit and code for the UI of a "Wizard of Oz" prototype of the v4 machine. This machine will be the successor of the machine, that is currently stationed at customer sites. The prototype will be used for presentations in front of investors. The interface consists of four buttons, a set of LEDs and an LED strip. Each button has the ability to light up, thanks to their inbuilt LED. The other separate LEDs, also called light indicators, each light up, depending on the current machine state. Here, the machine state is only determined by the UI. This will change in the working prototype of the machine. The LED strip on the is able to address each LED individually. The LEDs of the strip can change colour and brightness. The button LEDs and the light indicators can only change their brightness.

Once the user has inserted the ice cream capsule in the capsule holder, he can chose the firmness level of the ice cream. After the firmness level has been chosen, the user presses "Button 1", to make the holder ascend. After the holder reaches its final position, the ice cream is being blended. After the blending is completed, the first button lights up again, signalling the user to remove the capsule cap. When the user presses button 1 again, the ice cream is being dispensed. Once the dispense is complete, button 3 and button 4 light up as well. The user can press button 1 several times, after each dispense cycle, to dispense any remaining ice cream inside the capsule. If the user wants to finish the process, he can press button 4, to descend the capsule holder and dispose the capsule. [1]

2.1 Subtask 1: circuit design

The circuit consists of two parts. The first part consists of the four mechanic buttons. The buttons are connected to the five Volt pin from the board. The other end is each connected to the respective DIN pin. The buttons' inbuilt LEDs are each respectively placed on their left side. The circuits are shown in figures 3 and 4.

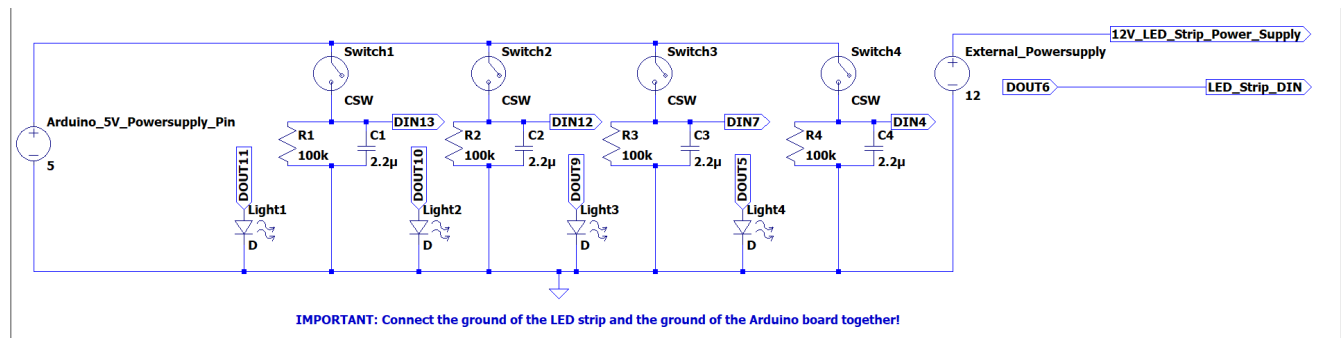


Figure 3: Circuit diagram of the interface in LTspice

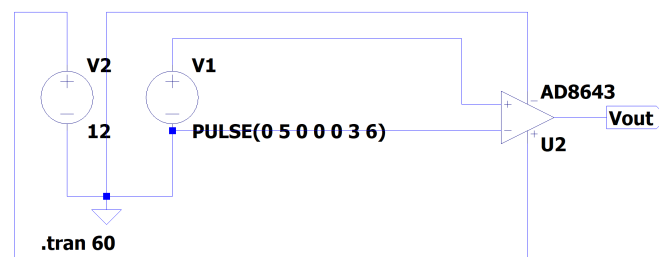


Figure 4: Capsule lighting circuit diagram in LTspice

Samples with the Arduino serial plotter have shown that the buttons output is not stationary, when the button is not being pressed. The output flickers between 1 and 0 in this case. This malfunction is the result of white noise. The DIN pin of the Arduino board detected a constant flickering from value true and false (1 or 0).

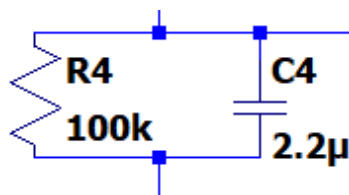


Figure 5: Low pass filter constructed from resistor and capacitor

To solve this problem, a capacitor and resistor have been placed parallel to each other. One end is connected to the DIN pin, that belongs to the microcontroller, and the button's output. The other end is connected to the ground. These components form a low pass filter. The capacitor is being used to flatten out any white noise that might interfere with the digital signal. The resistor creates a voltage divider between the board's 5V pin and the DIN pin. The resistance of the open switch has been observed to be way higher than 100 k Ω . Tests showed that a resistance of 100 k Ω is enough to keep the voltage low when the button is not being pressed and keep the voltage high when the button is being pressed.

$$\frac{U_{out}}{U_{in}} = \frac{R_{button}}{R_{button} + R_{resistor}} \quad (1)$$

$$U_{out} \in [0V, 5V]$$

$$U_{in} = 5V$$

$$R_{button} \in [0, \infty)$$

$$R_{Resistor} = 100k\omega$$

The equation shows that the output voltage will either be at ground level or at the same level as the input voltage. This however is only the case, if the button's resistance is considerably greater than the parallel resistor. Including the capacitor, the transfer function looks like this:

$$\frac{U_{out}}{U_{in}} = \frac{R_{button}}{R_{button} + R_{resistor} + j * \omega * C * R_{button} * R_{Resistor}} \quad (2)$$

It is apparent, that the function resembles a low pass filter. For low frequencies, the function resembles the voltage divider. For high frequencies, the resulting output voltage amplitude is closer to zero. The influence of higher frequencies is thus being reduced.

2.2 Subtask 2: UI programming

The code is written in the Arduino IDE, using the language C++. It is divided into three parts. The first part initialises all global variables, such as the pin addresses for the LED strip and the buttons. The second part consists of the setup function. It is used to set each of the used pins either as input or output and tell the compiler which type of chip the LED strip has built in. The function also calls the function to calculate the fading matrices. These fading matrices are being used for the LED strip, to achieve a smooth fading effect between two colours. All fading matrices are being stacked together into the fadingEffect matrix. This three dimensional matrix stores all colour information for all LED at all colour change fades.

To use the LED strip, import the "FastLED" library. Each LED can be configured individually and thus has to be initialised in the program. To do so, set the number of LEDs, the strip has. Each LED is addressed and configured by its index within the LED array. The value, assigned to the index of the array is this LED's colour value. Each LED has each a red, green and blue channel. Each colour channel takes an integer, ranging from 0 to 255. Together, these values form a three dimensional vector, representing the total colour of the addressed LED.

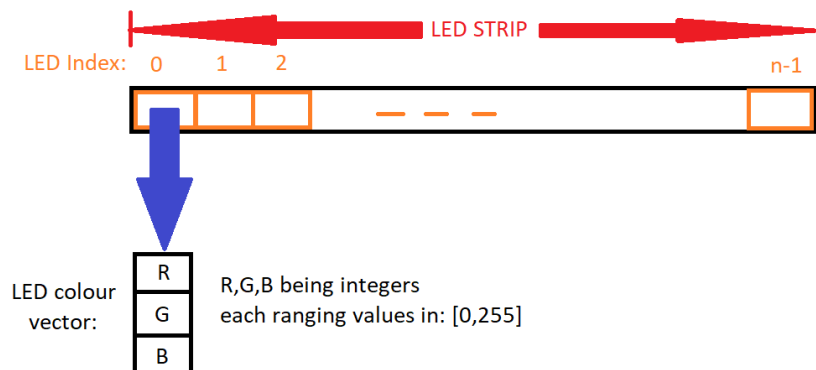


Figure 6: Sketch of the LED strip model, used in the program

To achieve a smooth fading effect, the code needs to know how the sequence looks like from a mathematical perspective. The fading effect matrix is used to store the information of all fading effects for each LED within the effect length. The total number of LEDs, the fading effect length, total number of colours and the fading speed can be chosen by the programmer. These variables are parameters, which will be used to personalise the fading effect. The resulting matrix is a three dimensional matrix. The matrix size is determined by the number of colours and the fading effect length. Figure 7 shows an abstract representation of the matrix.

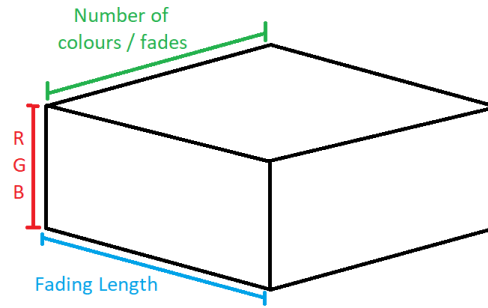


Figure 7: Sketch of the fading matrix dimensions

Within the setup function, the function to calculate the fading matrices' values is being called. This function iterates through all colours, that have been chosen by the programmer. Within this for loop, the colour, the strip has before each fading effect and its successor colour are taken from the colours matrix. The start and end values are being linearly interpolated within the second for loop. The linear interpolation formula looks like this:

$$x(k) = \frac{k * x_n - (n - k) * x_0}{n} \quad (3)$$

$n \in \mathbb{N}$

$k \in [0, n]$

The values are interpolated for each colour channel (RGB) from the first point of the fading effect length to the last. This is being repeated for all sequences.

The calculated values are being used in the functions used for the top to down fade and bottom to top fade effects. Both fading effects are divided into three cases, each with a for loop inside. The for loops iterate through all LEDs within the fading effect length and assign the desired colour values to each of the addressed LEDs. Each time, the values have been updated, the light output of the strip is being updated with the command "FastLED.show()". Before iterating to the next step, the controller waits for a certain amount of milliseconds.

The three cases are each being called iteratively within the functions main for loop. The top to down fade iteratively updates the fading effect, coming from the first LED to the last LED.

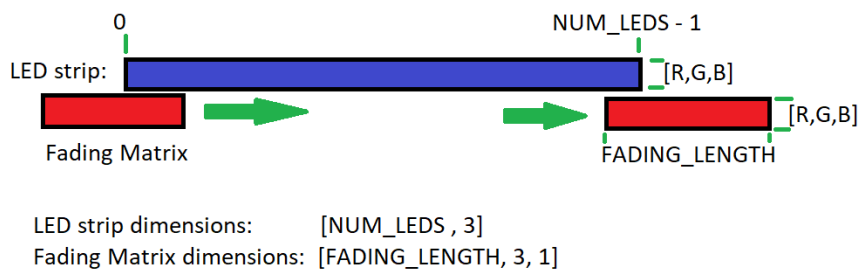


Figure 8: Fading effect matrix iteratively passing through the LED strip and adjusting the colour values

The bottom to top fade starts from the last LED and iteratively goes to the first LED. The three cases are similar to the three cases that occur, when doing a convolution over an array with a discrete filter. The first case is being executed, when the fading matrix has not completely entered the LED array. As soon as the fading matrix has passed fully inside the array, the second case is being executed. When the fading matrix can not completely fit inside the LED array anymore, the third case is being called. If the fading matrix and the LED array are no longer overlapping, the fading effect is completed. The top to bottom fade effect works the same way as the bottom to top fade. In that case, the matrix travels in the opposite direction. The general workwise is still the same.

The main function "loop()" is the protocol that the board is executing repeatedly. This function repeatedly reads the values of "button1" and "button2". The values are being passed to the function "processingFlow".

The processing flow function first checks whether button 1 or button 2 have been pressed. If not, the machine stays in the standby state.

Before the dispense sequence has been started (button 1 not been pressed previously), the user can adjust the firmness levels. Every time button 2 is pressed, the function to adjust the firmness level is being called. The function currently only updates the lights of the firmness level LEDs. Every time the function is being called, the light of one LED cycles to the LED of the next lower firmness level (e.g. regular to firm, firm to soft, soft to smoothie and smoothie back to regular).

To confirm the firmness level and start the dispense, press button 1. Button 2 LED turns off and the strip fades from black to white, travelling from bottom to top. In the final product, the capsule holder will ascend in this step. After the capsule holder reached its highest point, the capsule light LED strip is turned on. The machine then starts blending the ice cream, stored within the capsule.

After three seconds, the LED, signalling the user to remove the capsule cap, is turned on. This signals the user that the blending process is complete. As long as button 1 is not being pressed again, the machine remains in a standby waiting state.

As soon as the user presses button 1, the machine starts the ice cream dispense. The remove cap LED is turned off again and the colour of the LED strip fades from white to pink from top to bottom, indicating the dispense progress. After finishing the dispense, button 3 lights up.

The user can repeat the dispensing sequence by pressing button 1. During the additional dispensing sequences, button 3 is turned off. Button 3 is turned on again after completion of each extra dispense cycle. To finish the dispense process, press button 3. The capsule holder will descend again. The capsule lighter LED strip and the light of button 1 are turned off. The LED strip fades to black from top to bottom, with the same speed as the capsule holder descends. The firmness level is being reset and the machine returns to the standby state.

2.3 Subtask 3: hardware implementation

To make the lighting look as pleasing to the user as possible, a few refinements have to be done to the hardware implementation. The goals of the lighting are:

- Having a smooth fading effect for the LED strip
- Even lighting of each element without any spot lights
- High contrast between light and shadow when the LEDs of the strip and buttons are turned on / off / dimmed

The issue with LEDs is that they are dispensing light like spot light, up close. This makes the desired fading effect of the LED strip look rigid. Additionally, the LEDs for the lettering tend to not fully light up the whole area when set too close to the gap, through which the light should pass through. This leads to the labels not being fully legible.

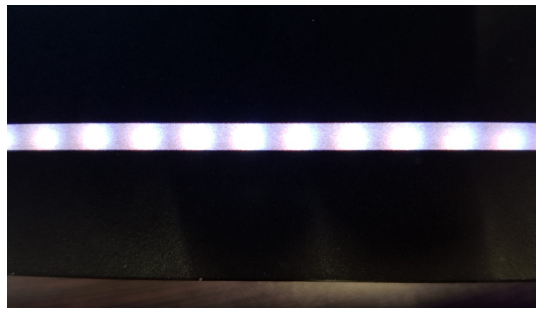


Figure 9: Problem shown with the LED strip

The first two goals can be solved in two different ways. The more intuitive solution would be to put a gap between the light source and the slit, through which the light passes through. This has been used for the single LEDs, assigned to the buttons. Following Huygen's principle, the further away the LED is being positioned to the gap, the wider the light is being evenly dispersed. Most of the light coming from the LED is concentrated in a conic shape, emitted from the top of the LED.

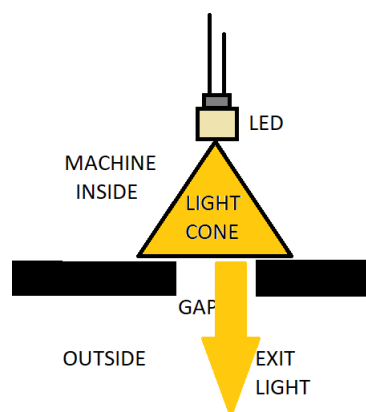


Figure 10: Diffused cone light passing through gap

The other solution would be using indirect lighting by reflecting the light on a rough surface. The rough surface will reflect the light and diffuse it. The rougher the surface, the more the light will be diffused.

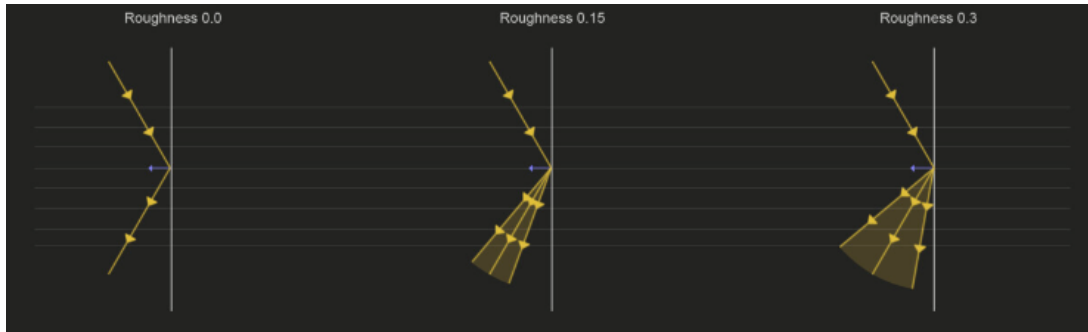


Figure 11: Diffusion of light in relation to surface roughness [17]

The first solution has been used for the implementation of the LEDs of the lettering of the softness values and the lock symbol. The LEDs have a distance of one centimetre to the front panel. To prevent the interference of each light with its neighbours, partitions have to be placed between each two neighbouring LEDs. Without the partition, the light of one cone can pass through several gaps, making it look like its neighbouring LEDs are lit as well.

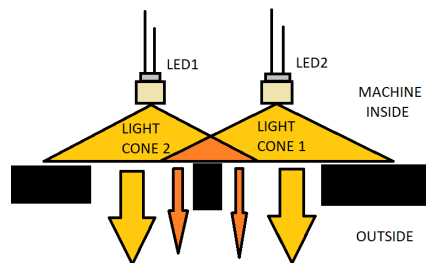


Figure 12: Influx of light coming from neighbouring LED

With the partition between the neighbouring LEDs, no light of each LED's neighbours will pass through its assigned light gap.

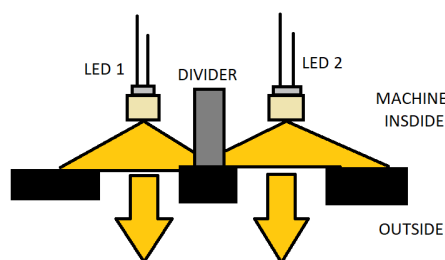


Figure 13: Influx of light limited by dividing wall between LED

The second solution has been used for the LED strip. The strip has been arranged in a 120° angle close to the gap. Together with the reflecting surface, the arrangement forms a triangular shape when looked at from the top.

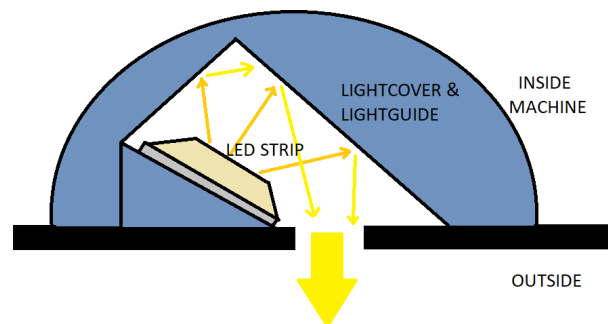


Figure 14: top view of the LED strip's light cover / guide

The results of both arrangements achieved the desired goals. Unfortunately, the process of the fading effect of the LED strip cannot exactly be shown in the pictures. However, the pictures show almost no visible steps in the fading and a high contrast between light and dark.



Figure 15: Smooth lighting from LED strip

3 Project 2: construction of the auger insertion mechanism

The auger insertion mechanism is used to ease the pre-processing step for the user. Currently, users are expected to drill the auger into the ice cream by hand. In the future, the auger insertion rig (AIR) will take over this task. This mechanism takes over this repetitive, manual step. By making the pre-preparation automated, the output is guaranteed to be the same quality for all future capsules. Additionally, the process becomes hands free. This is especially important for the food industry, in terms of hygiene and food safety.

The design process consists of five phases (see figure 16). The first 3D printed AIR prototype model already existed from previous development on this project. From this point on, the first code prototype is being developed. Complimentary to the code development, the sensors and UI components are added, completing the electrical development of the prototype. Code and electronics are being improved in iterations at this stage. Upon arrival of the PCB, the electrical design has been frozen. At this stage, the mechanics is being tested for any possible improvements before its design freeze. As soon as the sheet metal has been ordered, the mechanical design is being frozen. In the last stage, the prototype undergoes several testing phases such as robustness, life expectancy and output quality. If a problem arises, a change in software, electric or mechanics has to be made, before the product can be deployed on customer sites. In this stage, only minor changes to the hardware are meant to be made.

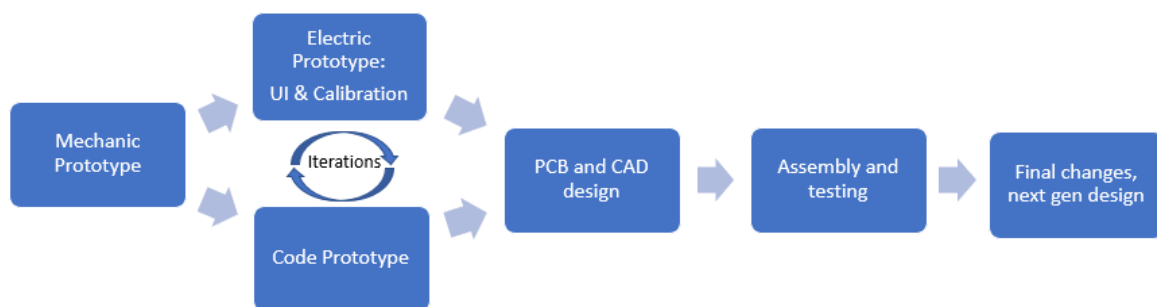


Figure 16: Development process stages for the AIR

Before further product development, a problem analysis has been conducted. The analysis consists of an assortment of possible error causes for the machine. The analysis will help in the following development steps. Knowing possible error sources raises awareness for finding their solutions in advance. The error analysis has been organised in a fishbone diagram. The fishbone diagram has the advantage to classify each possible error source to a cluster.

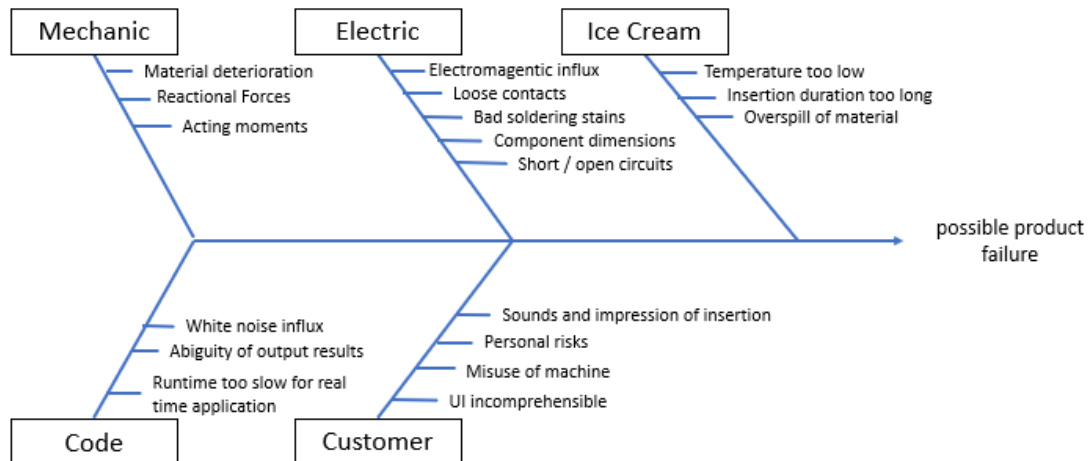
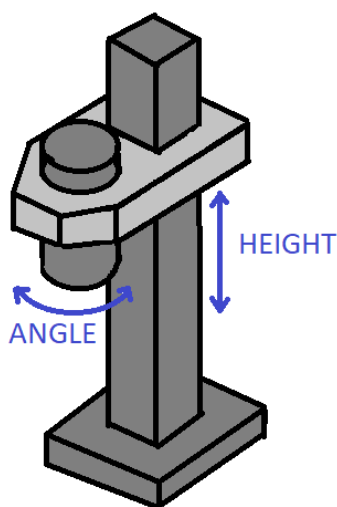


Figure 17: Problem analysis for AIR development

The error sources have been categorised into five clusters. Each cluster holds a set of possible causes for product failure. Dividing the product failure causes into clusters eases error analysis and assignment of possible problems to each section of the development process. For example, the problems listed in the cluster labelled "electric" have to be addressed when constructing the PCB (see figure 17).

The insertion mechanism consists of two joints, one used for the rotation of the auger, the other used for up- and down-translation of the first joint. Figure 18a shows a schematic of the machine. The schematic only shows the general work-wise of the machine. It does not match the dimensions of the final mechanical build. A picture of the prototype's first iteration is depicted in figure 18b. This prototype is later being extended by the limit switches and the elements for the UI and calibration.



(a) sketch of AIR

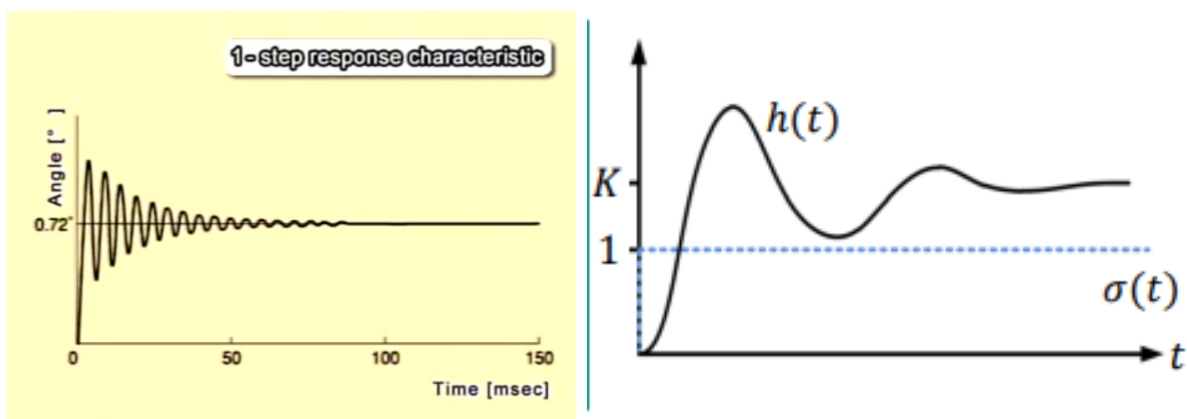


(b) Picture of AIR

Figure 18: Picture and Sketch of the AIR's first iteration

3.1 Subtask 1: noise reduction

The main issue, the mechanism had, was loud sound, coming from the rotation of the two motors. The sound was caused by vibrations of the motor, when it's rotor was moving. These vibrations are a result of the motor's impulse response.



(a) qualitative motor's step response [4]

(b) periodic step response of a PT2 element [5]

Figure 19: Comparison of step response of a PT2 element to the response of the motor

As shown in the figures 19a) and 19b), the step response of a stepper motor resembles the step response of a PT2 element. The cause of the motor's vibration is the rotor's periodic over- and under-shots during the step response. This over- and undershooting causes the vibrations, which later produce the loud sound, as soon as several steps are being taken in quick succession.

Since a stepper motor usually works with steps and impulses, the vibration can not be fully avoided. The most critical over- and undershot swings are the first few in the beginning. If they can be damped, the overall sound would be reduced significantly.

To control the motors, the board needs a driver which converts the digital signals and the PWM of the board into a signal, which the motor can actually use. The used stepper motor drivers are a version of 4 Quadrant Chopper, similar to the circuit shown in figure 23. The drivers are able to convert a DC voltage into an AC voltage, which is used to drive the 2-phase stepper motor.

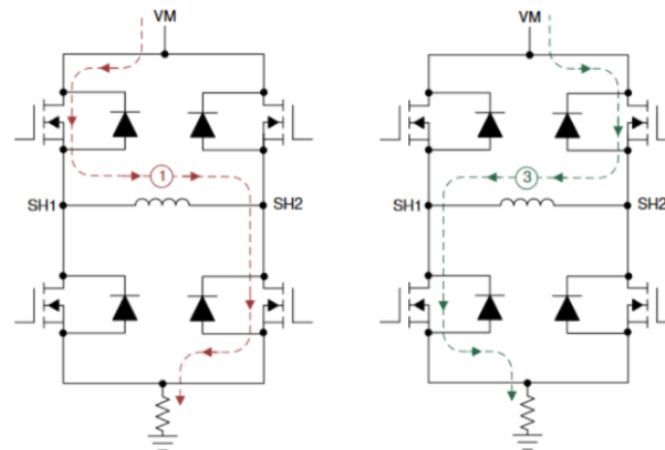


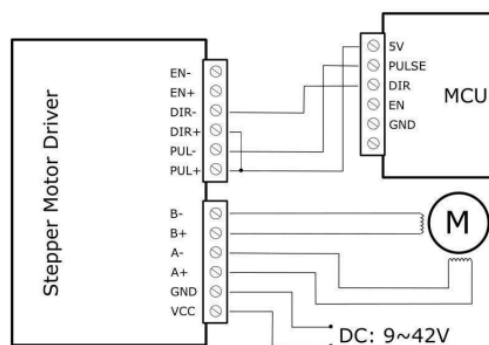
Figure 20: 4 Quadrant Chopper current flow for positive and negative output current. The Output current is the current that flows through the induction in the middle, which represents one of the motor coils [6]

There are several ways to fix the noise problem, that comes from the vibrations. There was a set of three possible solutions that were taken into consideration:

- Altering the Input of the motor by a controller. This requires a measuring unit of the motor's position in degree
- making smaller steps in rotation with an overall smaller amplitude of the input steps
- Increasing the speed, in which the input steps are being fed into the motor input

The schematic diagram, on how the controller, motor and board are connected are shown in figure 21. The configuration of the controller can be found in the appendix (Figure ??).

This is an example for the common-anode connection. ("EN" not connected)



Note: Please cut off the power when you connect the system, and ensure the power polar is correct. Or it will damage the controller.

Figure 21: Wiring of the controller, motor and board to each other [7]

The controller, that has been used for the AIR, is the TB6600. The machine is being controlled by an Arduino UNO. The used motor model is a stepper motor from the Nema 17 family (Figure 22).



Figure 22: Nema 17 Image [18]

At first, constructing a controller looks like the best decision, to solve the noise problem. It dampens the over- and under-swings that come from the jump of the input step and also has the potential of completely cancelling them out. The motor will need an encoder to track the position. The encoder is being used, at first to calculate the system parameters by measuring the motor's impulse- and step-response. Secondly to calculate the difference between the process variable and the desired setpoint. The controller would have to be implemented in between the output of the 4 Quadrant Chopper and the input of the motor. This would require the board to create an analogue output, which would directly go into the input of the motor according to the value, that the Chopper has set.

The second approach is a lot easier to implement and has most of its effect on motors that are being used with a low RPM. Most stepper motor drivers have choppers that can do "microstepping". The micro-step function makes the chopper to do smaller steps in between one complete step.

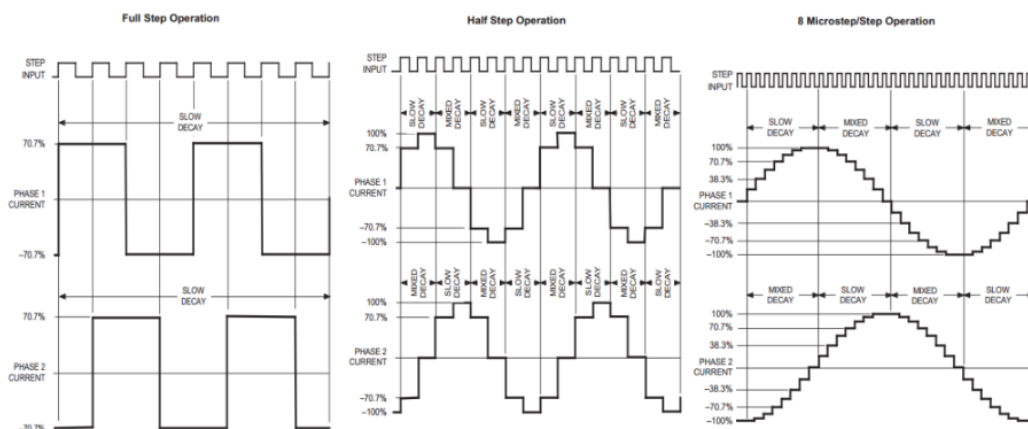


Figure 23: Output current of the Chopper depending on the amount of microsteps [6]

As seen in the figure above, the output of the chopper becomes more and more similar to a sine wave function. Furthermore, the amplitude of the steps is getting significantly smaller with the amount of microsteps. Since the intensity of the over-swings and under-swings is proportional to the amplitude of the step, the vibration will become way more damped.

The TB6600 can do up to 32 microsteps per whole step. Tests have shown a significant improvements of noise reduction, after this solution has been tried out. This solution achieves the desired goal while having little to no loss of available motor torque. The noise has been reduced from an initial 80 dBa to a level of 60 dBa. This improvement has proven to be so effective, that it later has been chosen as the implemented solution.

The last approach on the list can be implemented without having to change anything on the hardware. However, without any other measurements (e.g. microstepping), the effects will only be visible when driving the motors with a higher RPM. Figure 24 shows the decreasing effect of step response swings, the higher the motor's RPM is. This means that stepper motors are generally quieter at high speeds, compared to low speeds.

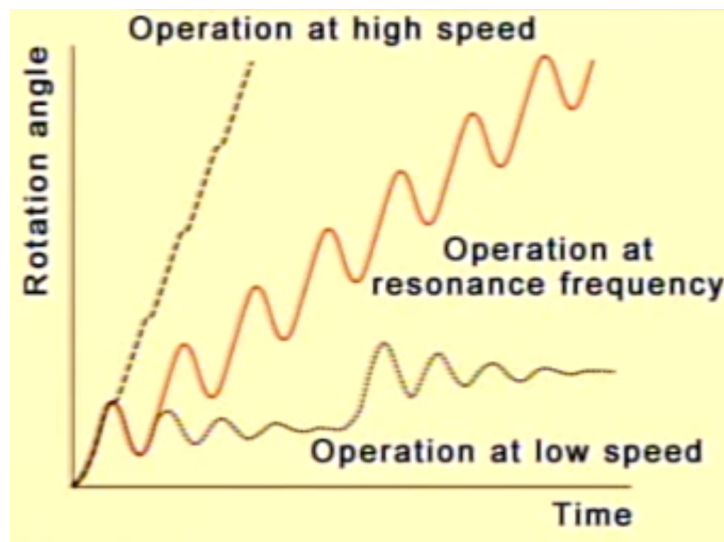


Figure 24: Continuous step response characteristics of the stepper motor at different speeds [4]

As seen in the image, the oscillation around the desired rotation angle becomes less and less apparent with higher RPM, since there is no time for the step response to decay, before it is overlapped by the step response of the following step. In conclusion, you won't need any controllers or microstepping as long as the RPM is high enough. In this case, the driver is still set to the maximum amounts of microsteps, since the motor also needs to accelerate and decelerate. To make these processes also as quiet as possible, the microstep still needs to be used.

To conclude the project's first part, the code for the PWM has to be written. A constant PWM signal will be enough to test the noise output of the motor. There are two ways to do so. The first way would be to program the PWM with the standard PWM library of the Arduino UNO. The library enables the board to set a duty cycle from 0 to 100 percent, for a fixed frequency. The

speed / RPM of the motor is controlled by the frequency of the PWM. Being able to control the frequency is especially important to accelerate and decelerate the motor. While the library enables an easy handling of the duty cycle, changing the PWM frequency is rather difficult. Since a PWM is just a repeating signal of high (5V) and low (0V) voltage, it can also be implemented by delays after a certain voltage - high or low - has been set. The delay duration for each value in each period is being used to set the duty cycle and also the frequency. Since the controlling of the frequency is essential for the controlling of the RPM, the second option is better for this case.

Quellcode 1: PWM Signal for the driver

```
1 //defines pins for Arduino UNO
2 int stepPin = 11;
3 int dirPin = 10;
4 int enPin = 9;
5
6 void setup() {
7   // Sets the two pins as Outputs
8   pinMode(stepPin, OUTPUT);
9   pinMode(dirPin, OUTPUT);
10  pinMode(enPin, OUTPUT);
11  digitalWrite(dirPin, LOW);
12  digitalWrite(enPin, LOW);
13 }
14 void loop() {
15   digitalWrite(stepPin, HIGH);
16   delayMicroseconds(4);
17   digitalWrite(stepPin, LOW);
18   delayMicroseconds(11);
19 }
```

The code shown above is the testing function for the motor noise. The pins, used for the digital input to the motor driver, are the digital GPIO pins 11, 10 and 9. The loop function plays one period of PWM cycle on repeat over and over again. First, the output of the step pin is set to high for a certain amount of microseconds and then set to low for another amount of microseconds. The ratio of high and low determines the duty cycle of the PWM. The sum of the microseconds determines the total periodic duration. The inverse of the periodic duration is the PWM frequency.

$$f = \frac{1}{T} \tag{4}$$

$$T = t_{low} + t_{high} \tag{5}$$

To sum it all up, the motor has been connected to the stepper motor driver, as shown in figure 21. The microstepping function has been chosen as the final solution to the noise problem. The

implementation of a controller was not feasible and would have taken too many resources. The third approach has not been examined further. It has been used in the later stages of the development, but not particularly because of noise reduction. It is merely a byproduct coming from making the machine operate as fast as possible.

3.2 Subtask 2: auger insertion control

Before any further works on the machine's mechanics can be done, the program had to be tested on the prototype. The main task of the program is the control of both motors. The auger will have to go as smooth into the ice cream as possible. The program for the PWM from the last subtask has been used as a basis for this program. The program has been extended by another motor and his respective GPIO pins for the encoder on the board. The circuit, with which the second motor has been connected to its own encoder, is also the same as the circuit for the first motor, shown in the recent subtask. Furthermore, a security button has been added on the top, to detect, when the linear joint has reached its highest position.

The setup function works the same as in the recent code. It has been extended by the pins for the limit security buttons, the second motor and the UI. The UI will be described in further detail in the next chapter. The looped main function, which is the protocol for the machine, is divided into four phases. When the machine is turned on, the linear joint moves the platform with the second motor upwards, until the top security button has been pressed. This is the the highest position, the platform can reach. As soon as the signal to start the insertion is given, the insertion mechanism prepares the actual insertion. The signal comes from the machine's UI.

During the insertion, the platform moves down a minimum amount of steps, until the bottom limit switch is being pressed. Meanwhile the second motor turns the auger. The minimum amount of steps are the amount of steps, spent for the acceleration of both motors. To finish the insertion process as quickly as possible, the motors start with a low RPM and then accelerate to the higher RPM. Without the acceleration, the motors would become unsynchronised and stop turning at higher PWM frequencies at the step pins. The lower the PWM frequency, the less likely this is going to happen. This comes at the expense of speed.

The insertion should be finished as quickly as possible. If the insertion takes too long, the ice cream's temperature might rise too far. This affects the dispense with the SwirlGO machine and worsens the end product's quality. Process duration becomes extremely important for customers that need to serve a large quantity of ice cream portions in a short amount of time. Customers such as retailers, restaurants and food court stalls mainly prioritise speed and consistency of their machines.

The machine enters the third state, as soon as the bottom limit switch is being pressed. The platform moves up again, without the second motor turning this time. The platform travels upwards until the top limit switch is being pressed. The machine enters the fourth state. The fourth state is a standby state. The standby state will be exit as soon as the user gives the signal in the UI to start the sequence again. When exiting the state, the machine immediately enters the first state. If the top switch is pressed, the machine instead switches to the second state.

To enable each motor, the "enabling" pin (enPin & enPin2) for each motor driver has to be set to high. Otherwise, the motor will not move, even if a PWM is being sent to the step input driver. The "direction" pin (dirPin & dirPin2) decides whether the motor will turn clockwise or counterclockwise. If the direction pin is set to high, the motor will turn counterclockwise. If the

pin is set to low, it will turn clockwise. This counts for both motors. The "step" pin (stepPin & stepPin2) is being used to control the PWM. Just like in the previous subtask, the PWM is being implemented with delays and digital writes.

The two motors turn with different speeds. The motors receive step inputs with different PWM frequencies. Both frequencies are controlled in the same program thread. Since the outputs of both pins are being controlled by hard coded commands, a composition of two frequencies has to be found. The resulting function should be ideally compact. If the code bloats up too much, it will become slow. This will affect the overall performance and speed of the machine. The worst case scenario would be that the PWM cannot be executed as intended. The code has to be able to execute the program in real time. Not being real time capable leads to an undesired PWM output to the encoder. This leads to undesired effects on the motor. To reduce the amount of code, the time for duty cycles (high and low separately) of the slower motor have to be a multiple of the faster motor's PWM period time. This reduces the implementation to a few for loops.

The code for one period of the slower motor looks like this:

Quellcode 2: PWM period for the slower motor

```
1  digitalWrite(stepPin2, HIGH);
2  for(int j = 0; j<3; j++){
3      digitalWrite(stepPin, HIGH);
4      delayMicroseconds(5);
5      digitalWrite(stepPin, LOW);
6      delayMicroseconds(10);
7  }
8  digitalWrite(stepPin2, LOW);
9  for(int j = 0; j<4; j++){
10     digitalWrite(stepPin, HIGH);
11     delayMicroseconds(5);
12     digitalWrite(stepPin, LOW);
13     delayMicroseconds(10);
```

First, the signal on the PWM pin for the slower motor is being set to high. The signal is held at high for three entire PWM periods of the faster motor, before being set to low for four whole PWM periods of the faster motor. In total, the PWM period of the slower motor takes seven times longer than the faster motor. The duty cycle is being decided by the ratio of completed periods of the faster motor between the high and low signal of the PWM period of the slower motor. In this case the values for the duty and period time are:

$$duty = \frac{3}{4 + 3} \approx 0.4286 \quad (6)$$

$$T = (3 + 4) * (5\mu s + 10\mu s) = 105\mu s \quad (7)$$

The ratio of both frequencies / period time (here: ratio = 7) and duty cycle have been discovered experimentally. With these parameters the auger turns with the exact speed with which it

is being drilled into the ice cream. For example: If the auger has to travel 7 cm into the material for one whole rotation, the linear motor has to travel 7cm downwards as well. It has to travel downwards with the same speed, that the auger takes for one revolution. This desired goal has been achieved with these values.

This becomes important as soon as the auger gets in contact with the ice cream substance. Achieving this goal will minimise the pressure, that is being applied between the auger and the ice cream, when it is being drilled into the material. The force, that is being applied on the normal is being applied to the machine, is a result of that pressure, with which the auger is being pressed against the ice cream. If the auger is turning too fast, the ice cream will over-spill and material will be wasted. However, if the auger is spinning too slow, more pressure will be applied on the ice cream. This will also increase the resulting force, at is being applied on the entire machine.

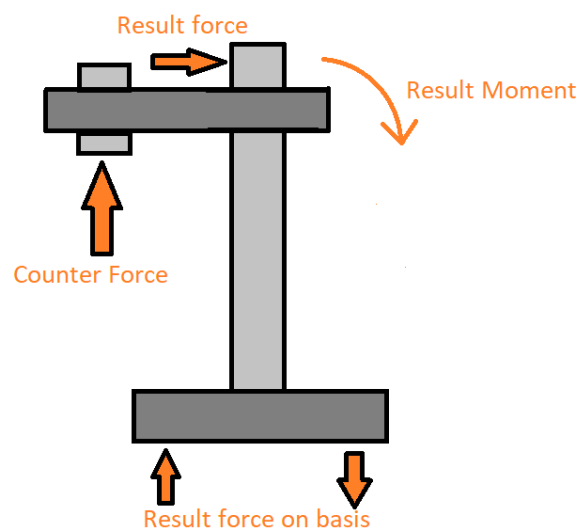


Figure 25: Resulting forces and moments, acting on the insertion rig (side view)

The problem originated from the tip of the auger. The tip of the auger is flat and therefore created pressure on the ice cream, as described before. Solutions would either be:

- altering the tip of the auger, so that there would be little to no surface directly on the normal to the move down direction. E.g. making the tip shaped like the tips for household drills.
- reinforcing the mechanism to withstand the feedback force. This will solve the problem for now but will not deal with the source of the force.

In this case, the second option is the more favourable option. Reason is the production cost for a new auger mould and casting of the new auger. The auger design has been frozen and thus cannot be changed in this case. The reinforcement has been installed on the back to counterfeit the moment on the machine's pillars, that comes from the applied force at the auger.

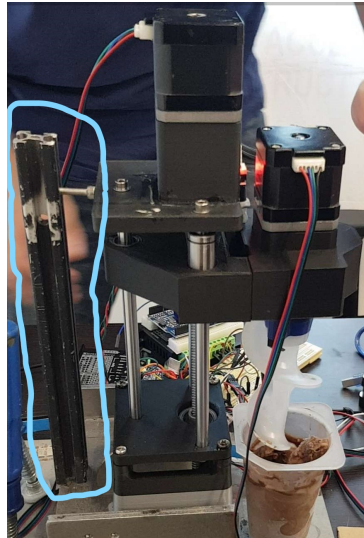


Figure 26: Reinforcement on the back of the machine

The reinforcement is marked in blue. The reinforcement provides additional stability on the back. It can resist the incoming force and thus reduce the stress on the rest of the AIR mechanics.

3.3 Subtask 3: UI design

The main functionality of the auger insertion rig has mostly been described in the previous chapter. This section goes further into detail about the UI design. The UI should be simple and robust. Additionally, the use of the machine should be as safe for the user as possible. To do so, the user has to be prevented from getting into close contact with any moving parts.

The UI consists of two start buttons, and a rocker switch to switch between the automated and manual mode. During manual insertion, the user will be required to press the two start buttons during the whole insertion sequence, each with its respective hand. Having both hands constantly occupied, the process is guaranteed to be continuously observed for the duration of the entire dispense and to be as safe as possible for the user.

If preferred, the user can choose to switch to the automated mode. This mode plays the sequence automatic and completely hands free, e.g. to do other task in the meantime. This automation mode can be selected by switching a rocker switch. To start the sequence, the user does not need to hold the start buttons. To start the automated sequence, the user needs to press both buttons and can let go at any time during the process.

The final element of the UI is the power switch. The switch can be used to turn the power on or off.

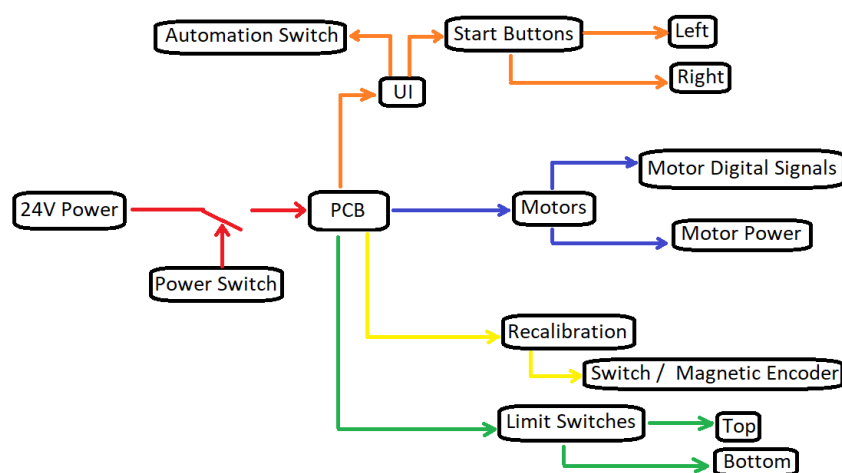


Figure 27: System chart of the auger insertion rig

The chart above shows the general connections of signal- and power-lines. The main machine parts can be divided into the 24V power source, the PCB, UI, motors, calibration and limit switches. The power source is directly connected to the DC-DC step down converter for the PCB and the stepper motor drivers. The DC-DC step down converter transforms the 24V to 5V. The power, supplied by the DC-DC step down converter, goes to the PCB's power input. The board's 5V pins and digital pins are each connected to their respective components. These components are the UI switches, the limit switches, the calibration and the motor driver's digital input. The motor drivers supply the motors each with a two phase AC power.

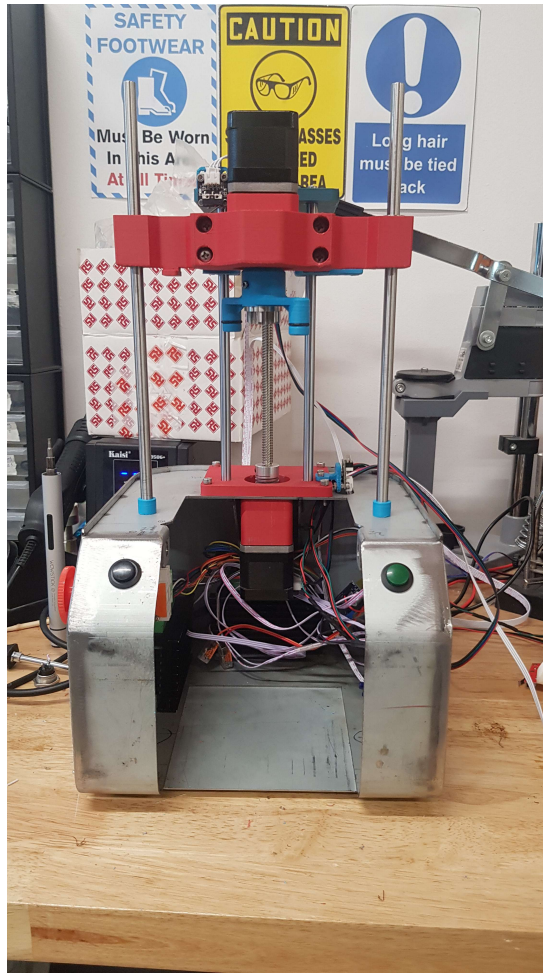


Figure 28: Picture of the first finished prototype with UI

Figure 28 shows the first functional prototype. The two start buttons can be seen on the front (black and green). The power switch and the automation switch are located on the back of the machine. An emergency button is located on the machine's left side. Pressing this button will turn off the machine. The motor drivers, PCB and stepdown converter are all located inside the machine. The limit switches are located on the same positions as in the last prototype. The calibration switch has not been implemented yet.

3.4 Subtask 4: PCB design

Before the machine can go into series production, a PCB and a mechanical 3D model of the machine have to be designed. This section goes further into detail about the design and construction of the PCB.

The PCB is a means to uniform the electrical connections within each series of machines. The breadboard prototype created a challenge when it came to the quality and repeatability of the machine's performance. Contacts often became loose and wore out quickly. This often led to an occurrence of short and open circuits. These problems were being addressed in the problem analysis at the beginning of this project and the found solutions have been implemented into the PCB design.

The biggest problem of the breadboard prototype is that the connections wear out quickly. Over time, cables, pins and connectors of elements become loose and thus not conduct electricity properly. This poses a problem for future deployment at customer sites, where transportation to site and usage can wear down the machine, making these problems occur. The PCB should be able to serve the same functionality as the breadboard prototype while having ports for all cable connections, that are able to hold the cables in place firmly.

Figure 29 and figure 30 show a comparison between the cable management of the breadboard prototype and the PCB. The cables have been cleaned up significantly. The circuit is more transparent and understandable in the PCB.

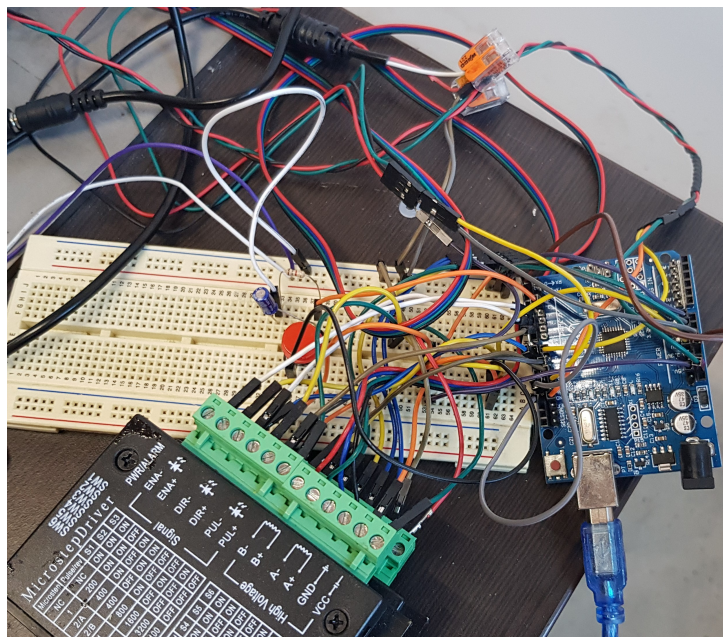


Figure 29: Cable management of breadboard prototype

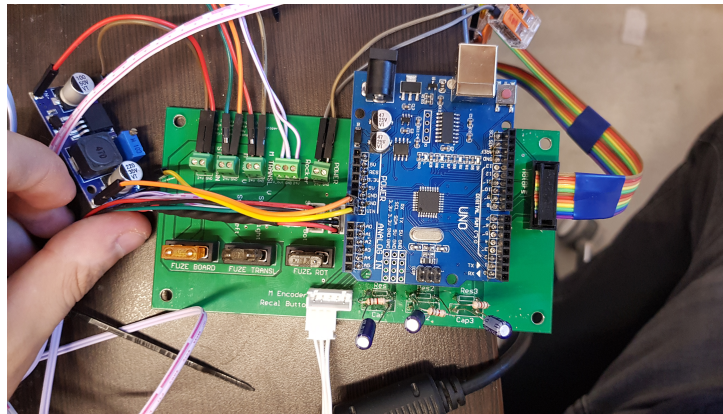


Figure 30: Final result for the PCB

The second biggest problem, is the lack of transparency, as seen in figure 29. The connections were unorganised and quickly assembled together throughout the early stages of the design process. It made the circuit nontransparent, which made it difficult to trace back errors. The traces have been cleared up in the PCB. This eases machine maintenance and guarantees the same quality over all machines of the same generation.

This becomes equally as important for the mechanical implementation of all electrical components. The placement of the electrical components also has to be planned in CAD. Therefore, a 3D model of the PCB has to be provided as well. Because of space limitations within the machine, the PCB also has to be as small as possible.

Additional requirements are a minimum cost for the construction of the PCB and its components. All of this combined with the delivery fees and delivery times have to be considered for the documentation of the BOM. The key cost-deciding factors for the PCB are the amount of layers it has and the thickness of the copper traces. The PCB uses two layers and the standard copper thickness of $16\text{ }\mu\text{m}$.

Each project is divided into four folders. The source documents folder contains the circuit and PCB file files. The data of all components is stored in the library folder. The data of all components within one library are stored in their respective library file. These library files are then stored in the library folder. Each component has their information stored in a PCB library and a schematic library. The PCB library stores the footprint of the component, while the schematic library stores the component figure. The documentation folder has not been used in this case. It is used to store documents, describing and explaining the design choices. The folder called generated stores all generated files, such as gerber files. The generated gerber files will be sent to the manufacturer.



Figure 31: File organisation in Altium designer

The application that has been used for the PCB design is Altium designer version 16.1 . Altium's advantage is, that it also creates a 3D model of the PCB. This model can be implemented into the CAD file. This eases planing the placement of all components in CAD.

At first, a circuit diagram has to be constructed within the software. The circuit is the same as the circuit that is constructed on the breadboard.

As soon as the work on the .SchDoc circuit file is complete, the information is being imported to the .PcbDoc file. After all changes to the file have been validated, the footprints of all components and all connections between each footprint pins are being loaded in by the program. It is then the user's task to arrange the footprints. The resulting traces should be as clean as possible. This includes, that the traces should be short and simple.

The main elements of the PCB are the footprint of the Arduino board, the digital cable connectors for all buttons and switches, the digital outputs for the motors and the connectors for the power supply (figure 32).

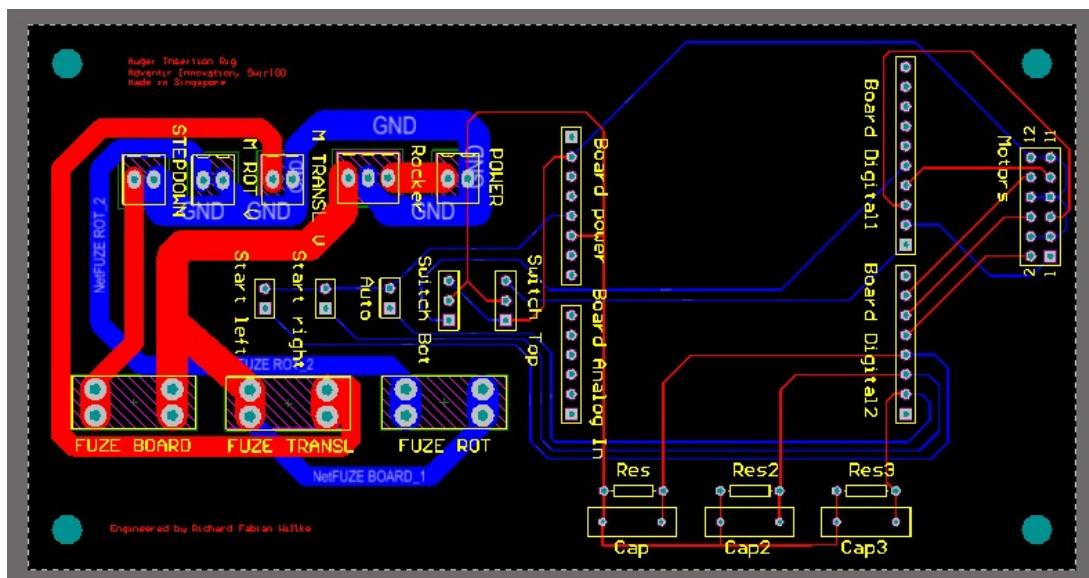


Figure 32: First PCB milestone

The traces and footprints for all power supply system components can be found on the far left. They are significantly wider than the digital signal traces. The Arduino board's footprint can be found in the middle of the PCB. The space between the pin rows has been kept clear, because of the limited space underneath the board. The headers for all digital elements, such as the buttons and switches for the UI and the limit switches have been placed on the board's left. This part is framed by the traces of the power supply system. The header, assigned to the two motor drivers' digital inputs, is located on the far right.

As soon as the traces have been laid, the trace width for each connection has to be set. The more current a component drains, the wider its trace width has to be. An online trace width calculator has been used to calculate each trace width [13].

Input Data		
Field	Value	Units
Current (max. 35A)	3	Amps
Cu thickness	16	μm
Temperature Rise (max. 100°C)	10	°C
Ambient Temperature	25	°C
Conductor Length	1	inches
Peak Voltage	24	Volts

Results Data		
Internal Traces		
Trace Data	Value	Units
Required Trace Width	7.78	mm
Cross-section Area	0.12	mm ²
Resistance	0.0037	Ω Ohms
Voltage Drop	0.0112	Volts
Loss	0.03	Watts

External Traces		
Trace Data	Value	Units
Required Trace Width	2.99	mm
Cross-section Area	0.05	mm ²
Resistance	0.0097	Ω Ohms
Voltage Drop	0.0291	Volts
Loss	0.09	Watts
Required Track Clearance	0.71	mm

Figure 33: Interface of the trace width calculator

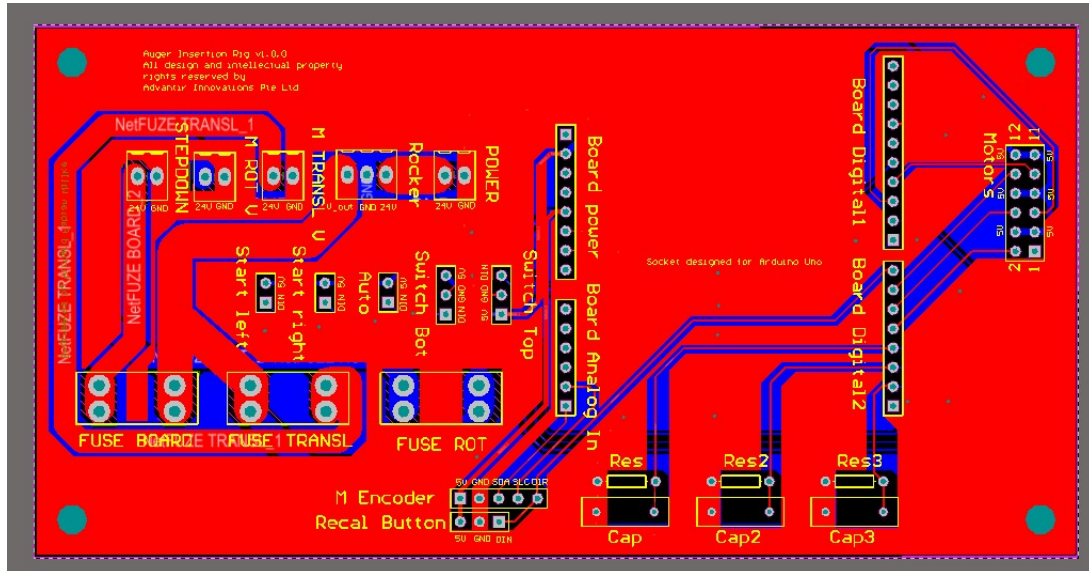
The copper thickness is fixed at 16 μm . It is the standard thickness for most manufacturers around Singapore. The power supply supplies a voltage of 24V. The maximum current, the motor drivers can tolerate, is around 5A. The DC-DC down-stepper can operate with currents up to 3A. The values have been considered in the trace with calculation. Only the result for traces on the outside layers is being taken into account. The board only has two layers. There are no internal layers in this PCB design.

For digital signals, the trace width is being kept as the standard trace width of 0.1 inches. The digital components drain only a small amount of current, when put into comparison to the current, needed to supply the power source elements.

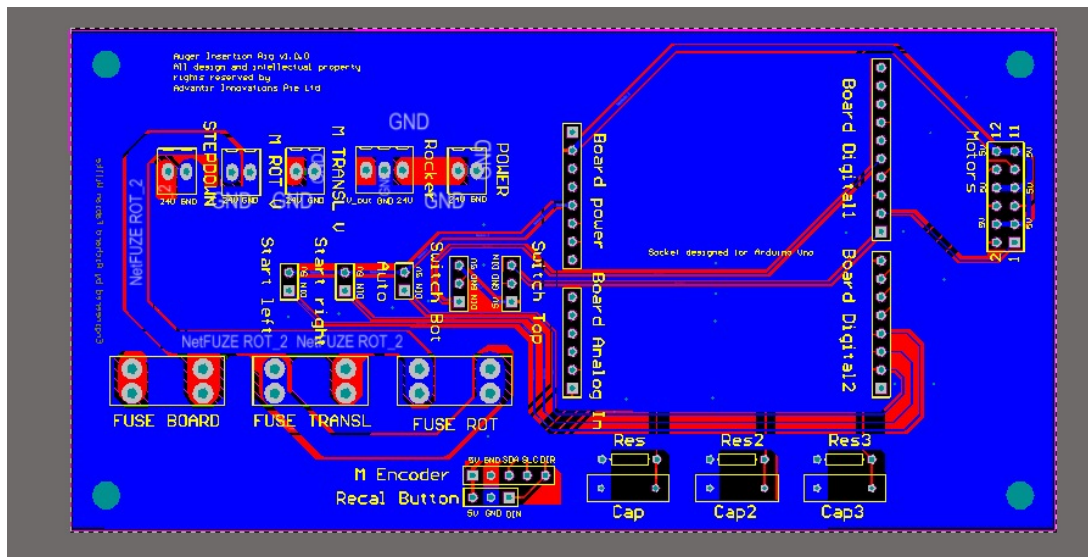
Following these steps, the PCB's robustness is being improved. A possible error cause within the PCB is electromagnetic influx between traces. These fields will interfere with the currents and voltages of neighbouring traces. These effects will be most noticeable on the digital signal traces. To minimise this influx, the digital signal traces should not cross ways with the power traces and should be separated far from each other.

The power and traces have been reshaped to achieve the minimum amount of crossings and the maximum distance between both trace types. Additionally, areas of copper, connected to the ground, are being placed on the empty spaces on the board. This creates an isolation around each trace, similar to the functionality of a coax cable. These areas will prevent the

signals of several traces to interfere with each other through their electrical fields. The results can be seen in figure 34.



(a) PCB traces top layer



(b) PCB traces bottom layer

Figure 34: PCB traces of top and bottom layer. Large areas of blue and red are areas of copper, connected to the ground

Figure 35 shows the difference between two traces either with or without a ground plane in between both. Without the ground plane, the traces will interfere with each other. The traces are interfering with each other because of the voltage difference between both. With the ground plane in between both traces, the traces are no longer coupled. This is because the voltage no longer falls of between both traces but instead from each trace to the ground. The effect can be seen in the schematics on the right side of the figure. Two neighbouring traces can be

approximated with a capacitor. In the second schematic, the potentials of both traces are no longer directly connected with each other, but instead directly to the ground.

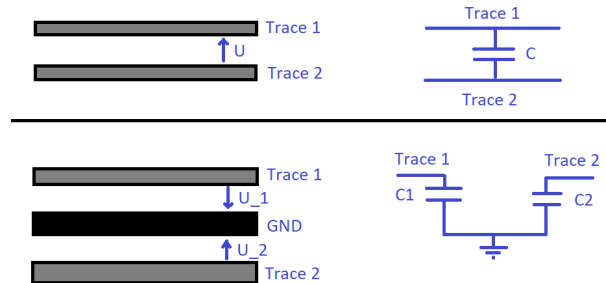


Figure 35: use and advantages of inserted planes connected to ground

Finishing up the PCB planing, the best fitting pin headers for each component cable connection has to be chosen. The headers should be able to hold each its assigned cable firmly in place. For the power connections, screw terminals are being used. The screw terminals have been proven to be robust and also being able to let high amounts of current pass through. The screw terminals each have a pinch of one inch. The pinch is the same as the pinch of all other pin headers on the board. If needed, the screw terminals can be replaced by any other header type, that has a pinch of one inch. This may be considered for costing down the electronics components. Screw-Down terminals have a higher cost than most other types of headers. To save delivery fees and material cost, it may be beneficial to change the screw terminals to one of the types, that are already being used on the board.



(a) 2 Pin clip connector [19]



(b) 2 Pin clip header [19]

Figure 36: 2 Pin connector and header

The board uses clip headers for the digital components. These headers keep the cables firmly in place and also makes plugging in all components easy. The orientation of the plugs and order of cables to pins are clear, even when plugging all components in and out several times. This makes disassembly and assembly easier, since the assembly and disassembly team does not need to know the designation of each pin with its respective partner, but only where which plug will have to go into.

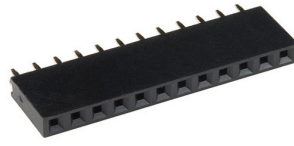


Figure 37: female header, used to hold the board [20]

The Arduino board will be mounted on rows of standard female headers, similar to the ones on the board itself. The mechanical forces on the board are minimal, compared to the ones on the cables to the board. Tests have shown, that the board is held firmly in place on the footprint.

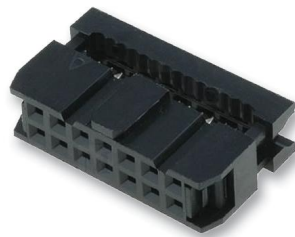


Figure 38: female header, used for the digital connections of the motor drivers [19]

The connection for the motor pins is a crimp pin connector. The header further cleans up the cable management by combining all cables into one multi cable line. The cables are crimped into the connector. The connector is clipped on the header. The other end of the cables are being connected to screw terminals on the respective motor drivers.



(a) Resistor [21]



(b) Capacitor [19]

Figure 39: Capacitor and Resistor used in the PCB

The resistor-capacitor circuit on the bottom of the PCB is being used to eliminate any remaining white noise, coming from the buttons and switches on the DIN pins of the Arduino board. The main functionality has already been explained in the chapter "Project 1". Because of the smaller trace width, the capacitors and resistors have to be sized down to smaller values. This is to guarantee that the capacitors do not store too much charge and that the charge can discharge quickly through the resistor. The resistor has to be large enough to fulfil the requirements. If both values would have been kept on the higher values, the discharge will take too long and cause delays between the UI and the machine output.

3.5 Subtask 5: BOM and documentation

The bill of materials (BOM) is used as a documentation for the construction of the machine. It is a means to have an overview all needed parts and track the overall cost of the machine. The BOM has the information about the name, quantity and price per part of each element. Additionally, the source is also being referenced for each component.

The BOM makes it easier to plan the overall expenses for the construction of the desired machine. For future generations, improvements in terms of costing-down are easier to achieve. The components, that have a cost, far more significantly impacting on the overall price than other components, can be found easier, when directly compared to the cost of all other components. In the AIR, the cost of the stepper motors is standing out the most. For future generations, optimising the stepper-motors to have a lower cost will be one of the first aspects that should be targeted.

3.6 Subtask 6: second generation PCB

This section deals with the design of an improved PCB. The new PCB should increase the quality of the electronics within the machine.

One of the previous generation's issues is the excessive use of space within the machine. Because of the PCB's size, it was not possible to place the 230V AC to 24V DC converter inside the machine. The second generation should be able to have the same functionality as its predecessor while taking up less space. Additionally, the price for the components is aimed to be lower or at max the same price as the previous version.

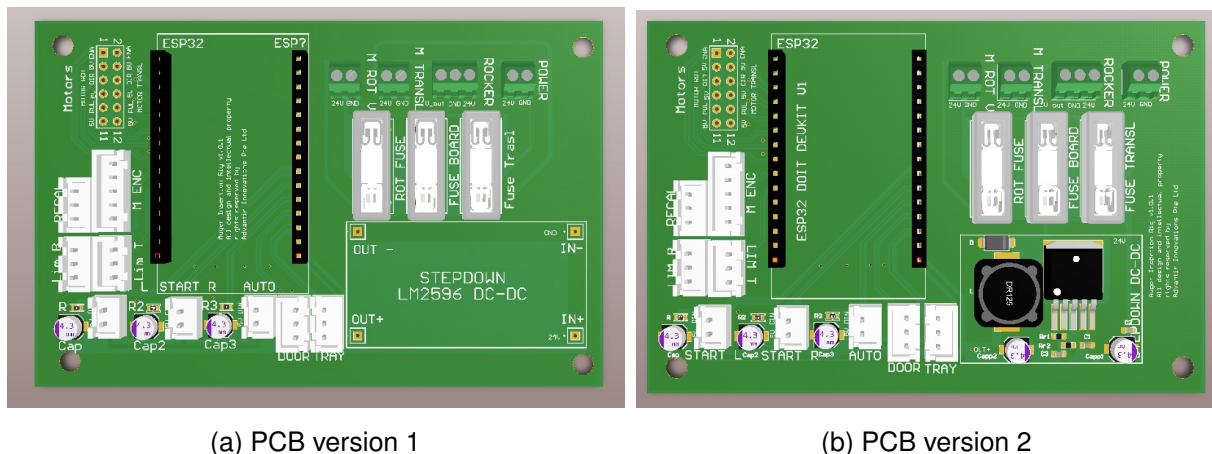


Figure 40: two versions of the second generation

Two designs have been developed. The difference between the two versions is the placement of the DC-DC stepdown converter for the microcontroller. In the first version, the stepdown converter is a separate component, that is placed on the board in the same fashion as the microcontroller. In the second version, the DC-DC converter's circuit has been reconstructed and its components soldered directly on the board.

The Arduino UNO board has been replaced with the smaller ESP32 Devkit V1. The Arduino took a significantly larger amount of space when being compared to the ESP. The ESP has the same capabilities while also providing more pins that can be used as digital input pins. Having more pins is also beneficial for improvements in the future. It allows to upgrade the machine with additional sensors. Currently, there are only two additional sensors being added to the machine.

One of them is a limit switch for the tray. The capsule, in which the auger will be inserted, is located inside the tray. The limit switch is used to detect, whether the tray is completely pushed in. If the tray is not pushed in, the insertion will not start.

The second sensor is a limit switch for a safety door. To prevent any object to get into the machine, the safety door on the front of the machine has to be closed before the insertion. If the door is open, the insertion will not start.

If either the door is being opened or the tray pushed out during the insertion, the insertion will be interrupted until both of their switches are pressed again.

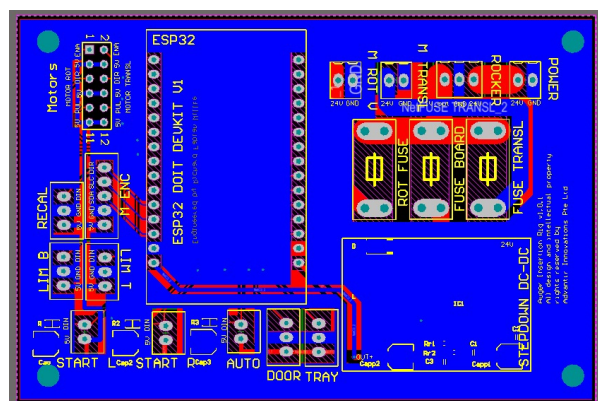
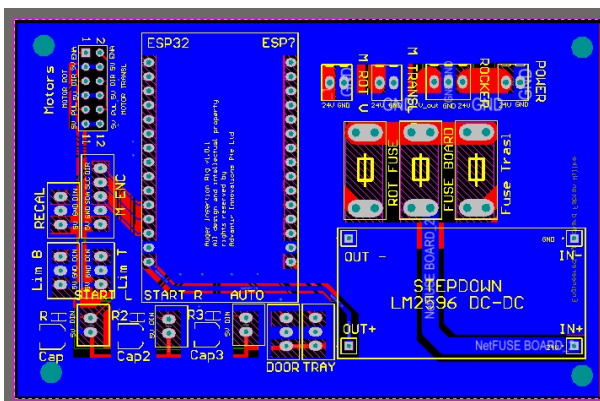
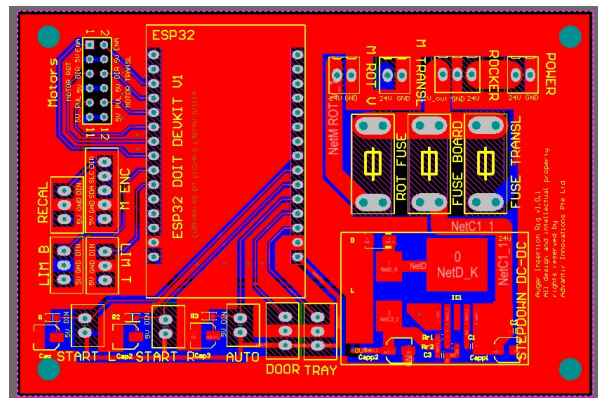
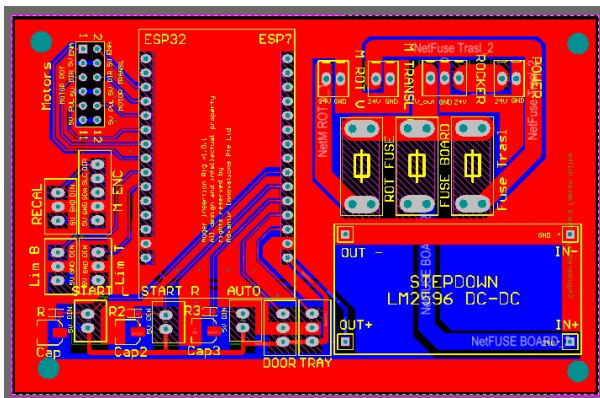


Figure 41: solders of the two versions of the second generation

Additionally, the previous on board resistors and capacitors have been replaced by their top solder component counterparts. These take up way less space and allow for a more compact placement. This also allows placing the resistors and capacitors closer to the ports of their respective switches. Furthermore, it makes the trace length as short as possible and thus also eases the understanding of the PCB.

The power lines and digital lines have been further separated from each other. The traces and ports, used for the power supply, are all located on the board's right side. The digital ports and traces are all located either under the ESP or to its left. Placing the power traces on this location allowed for a clear division between the part of the board, responsible for power supply and the part, used for digital signals. This also allows for a better placement of the traces. The new traces are shorter and easier to understand. They are also further away from the digital traces, thus further decreasing the possibility of influx between these traces.

4 Project 3: torque control for the motor of the v3.9.5 machine

Goal of this task is a cost reduction of the machine's hardware. Its most expensive components are the motor with its gear and power-supply. The current hardware consist of a Nema 34 hybrid servo stepper motor with a 3:1 gear. The configuration is able to hold a holding torque of around 22.5 Nm [74].

The main factors, affecting the price of a motor, are the type of motor, its holding torque and the maximum RPM. In this task, a set of motor, gear and power-source has to be found. This set has to be cheaper than the current setup while having the same output of RPM and holding torque.

The motor should be able to control its RPM and have enough holding torque to get the ice cream starting to move. The current motor is said to be oversized for the used machine type. The first parameter that is planed to be observed and optimised is the holding torque. If a lower power motor can be used for the application, the cost would be guaranteed to be lower than the cost at the time.

4.1 cost down

Research into the topic showed that there are two ways on how to approach the measurement of the minimum torque, needed to get the ice cream moving within the capsule. The first way would be to exchange the present motor with a servo motor, that has a torque control mode. With this motor, it is possible to observe, at which set torque the material within the capsule will begin to move with the turning of the motor. The setup has the potential to be used in several other machines, for the construction of prototypes or parameter optimisation. The search for fit hardware has yielded these three promising results:



Figure 42: Suggested motor-driver-sets

The quotes for the three sets can be found under the chapter "Literature". The second way would be the implementation of a rotary torque sensor. The torque sensor measures the torque between two points alongside the sensor's rotor's material. The torque

sensor can be implemented directly between the output shaft of the motor's gear and the auger.



Figure 43: example of a rotary torque sensor [12]

Rotary torque sensors are in the same price range as the previously presented motor-driver sets. The price scales with the durability of the torque sensor. The more the torque sensor can withstand in torque, the more expensive it is. The range, that is needed in this case, lies between 0 and 30 Nm. The rotary torque sensors can be implemented without having to change anything about the machine's preexisting motors, gears and their respective drivers and power-sources.

5 Project 4: ease of tests and machine usage

This section deals with the development and improvement of testing methods. The presented products are being used to ease the testing of the SwirlGO machine's performance. The main metrics that are being tackled are the uniformity of testing conditions and the comparability and qualitative evaluation of the testing results.

The uniformity of testing conditions makes test results comparable to each other. Under unified testing conditions, altering only a set of chosen parameters at a time, will also improve error analysis and optimisation of the SwirlGO machines.

To make the test results easily comparable, testers have to be constructed. These testers are used to yield the metrics, which will be used to evaluate the machine's performance.

5.1 cup spinner for testing

This machine is used to improve the comparability of ice cream dispense tests, by further uniforming the testing conditions. The problem, that had to be solved, was the manual spinning of the cup when dispensing the material. The consistency and quality of the dispensed ice cream was not only being evaluated by taste but also by looks. The main metric, to judge the looks quality is the ice creams firmness. The firmness affects the ice cream in how well the swirl can hold its form. If the swirled tower would fall into itself shortly after the dispense, it is a sign of bad dispense. The ice cream should ideally stand in a tower form shape and hold its form for a while.



Figure 44: Picture of a good sample

Since the positioning of the cup plays a significant role in the final result, the position has to be uniform for all iterations and tests.

The machine consists of one motor, that spins the cup on a plate during the dispense. To start the machine, the user has to press the start button and press it again to stop it. During the time of operation, the machine's motor spins with a constant speed. The cup sits on top of the motor.

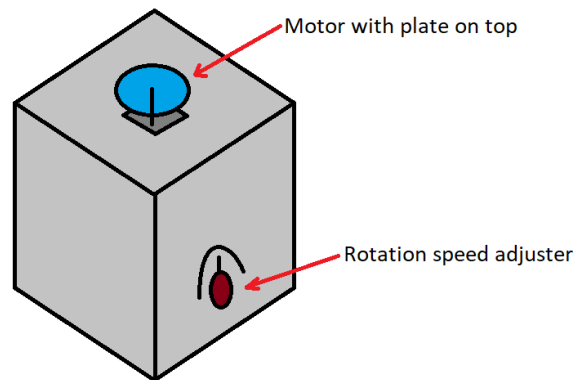


Figure 45: Sketch of the cup spinner machine

The machine's program is based on the basic PWM function of the auger insertion mechanism (see code: "PWM Signal for the driver"). The user can adjust the rotation speed by turning the knob of the potentiometer on the side of the machine.

To save space within the machine, the silent stepper driver TMC2209 V3.0 has been used. It is commonly used for 3D printers, when the motor doesn't need much torque. This driver is much smaller in comparison to the stepper motor driver, used in the AIR machine.

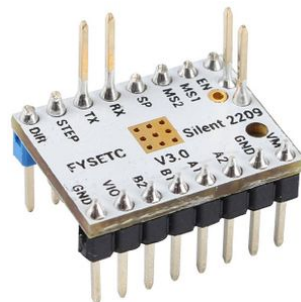


Figure 46: TMC2209 V3.0 stepper motor driver

For the motor power supply, a standard 12V output, plug in power supply has been chosen. The 12 V have been chosen because of recent tests with a 12 V stationary power source (Tests came from experiences with the auger insertion mechanism). The microcontroller takes 12V as a power supply input. Since the motor does not need to supply a high torque, the 12 V will be enough for this case, which is why the same power-source is being used for the power-supply of the motor.

The machine has not passed beyond the first concept stages. No complete prototypes were constructed.

5.2 video analysis of the fluid flow

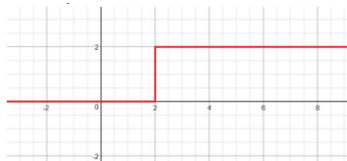
Currently, the analysis and evaluation of the ice cream flow, coming out of the capsule, has to be done manually. In larger quantities, this might become a problem. The manual evaluation takes a lot of time. every time a complete set has to been tested. Furthermore, each person evaluates the flow differently. Having clear, comparable metrics as test outputs will make test sets comparable to each other and give a precise output.

The metrics of choice are the width and colour of the ice cream flow, over the course of the dispense. The program will primarily be used to evaluate the flow's consistency and the homogeneity of mixed in materials within the ice cream.

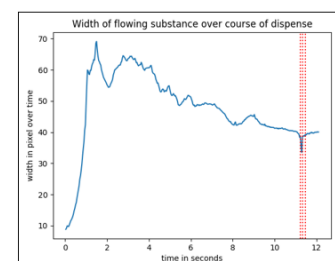
The mixed ins are smaller foods such as M&M's, crumbled cookies etc., that are later being mixed into the ice cream as a supplement. Being able to have mixed ins within soft serve ice cream is a difficult task for most machines. The common machines do not use capsules. This makes cleaning and maintenance more extensive. Having smaller particles within the material will make the cleaning even more difficult and thus make the material wear down quicker.

Being able to mix in mixed ins within soft serve ice cream will be the main signature of the SwirlGO machines. This is the main factor that will make the brand stand out from its competitors. This testing program is the first step to enable the success of this project.

The program's purpose is to analyse the ice cream's width and colour over the course of time during the dispensing process. The goals is to have the curves stay at a constant value over the whole dispense cycle. Ideally, the curves should each look like a step function.



(a) step function with time delay



(b) test result of flow

Figure 47: Comparison of ideal run with test results

The program is written in Python, version 3.10.0 . The program makes most of its functionality out of the use of the libraries openCV (in code: cv2) and tkinter. OpenCV is used for the analysis of the flow, using computer vision. Tkinter is being used to construct the program's UI. It is also being used to open and navigate through filesystems.

The libraries numpy and math are being used for mathematical and vector operations. These libraries are used to calculate the desired analysis function outputs.

Pickle and pandas are used to store data. Pickle enables the program to store previously customised data of the program's parameters. Pandas is used to export the program's output data as excel file.

Datetime is used to access the current UTC time. The UTC time is used to name the output

excel file at the end of the program's analysis part. The current date and time makes it able to give each file a unique name, preventing overwriting an already existing file.

Quellcode 3: used program libraries

```
1 numpy as np
2 import matplotlib.pyplot as plt
3 import math
4 import tkinter as tk
5 from tkinter import *
6 from tkinter import filedialog
7 import cv2
8 import pickle
9 import os
10 import pandas as pd
11 import datetime
12 os.system('cls')
```

The program consists of five functions. The most important function is the `analysisProgram()` function. The functions to update the canny edge detector parameters, the dimensions of the ROI and the function to upload the video path within the file system are all used to supplement the main analysis function. These functions store crucial information. The information will be passed to the analysis function.

The post analysis function takes the analysis function's output and uses it for further data analysis and improvement.

Quellcode 4: analysis program parts

```
1 def update_border_params()
2 def setROIborders()
3 def uploadVideo()
4 def analysisProgramm()
5 def postAnalysis()
```

The code is receiving a pre-recorded video of the flow as input. The video is being recorded from a documentation stand at one of the machines. The main metrics the lighting, contrast to the background and height of the camera have to be kept the same for all samples. This is necessary to ensure the comparability of all samples.

Pictures of the experiment stand are shown in figure 48. The camera is positioned in front of the machine and facing the space between the capsule mouth and the cup.



(a) Filming stand, viewed from side angle



(b) Video stand, viewed from front view

Figure 48: Video recording stand

The UI is divided into two main parts. The program's "Processing" part is used to analyse the flow and yield the analysis result's unfiltered data. The received information is shown in graphs, plotted against the time. The examined metrics are the flow's width, hue, colour saturation and colour intensity.

These plots show the raw, unfiltered data about the flow's width and colour. The program's second part consists of the so called "Post Processing" function. Its purpose is to provide the opportunity to clear the data of any white noise and thus making it able to receive more useful information out of it.

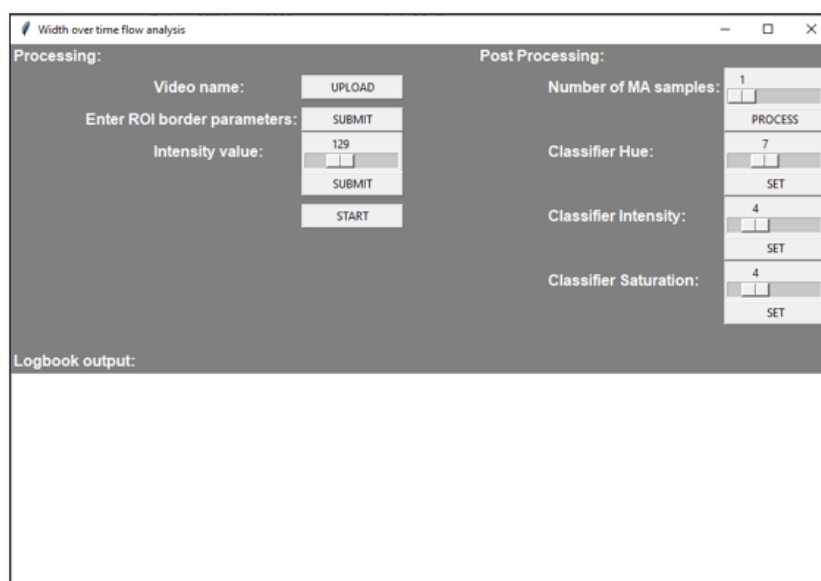


Figure 49: Flow Analysis Program Interface

The UI is being generated in the last part of the code. It consists of labels, buttons, sliders and text output fields.

All components are being placed within their assigned parent. In this case, the parent is the interface window. For clarification, it has been named window in this program. The main function of this program is called the mainloop. It runs as long as the UI window, it is associated with, is kept open. As a result, the program does not need another main function while the window is open. As long as the mainloop function runs, the user can interact with the program via interactive elements such as buttons, sliders and text entry fields.

Labels are being used to display text on the screen. Things such as the background colour, foreground colour and font can be set to the liking of the programmer. In this case, the foreground colour is set as white and the background colour as black. The font is called "none 12 bold". This colour scheme has been kept for all components except the buttons, sliders and output windows.

A button calls a function, once it is being pressed. The functions have to be declared in the same way as every other function.

Sliders are an other interactive element for the user. They can be used to set values. These values will be used in future functions.

Texts are fields, used to display text information. Here, they are used to give feedback to the user about the current state of the program.

Quellcode 5: interface main elements

```
1 window = Tk()
2 window.title("Window title")
3 window.configure(background = "sample colour")
4 window.mainloop()
5
6 Label(parent = window, text = "sample text", bg = "sample colour", fg = ←
   "sample colour", font = "sample text font")
7
8 Button(parent = window, text = "sample text", width = 14, command = ←
   functionName)
9
10 slider = Scale(parent = window, from_=0, to=20, orient=HORIZONTAL)
11 slider.set(4)
12
13 output = Text(parent = window, width = 108, height = 14, wrap = WORD, ←
   background = "sample colour")
14
15 anyComponent.grid(row, column, sticky)
```

To start the analysis, the user first has to select the video to be analysed within the computer file system. As soon as the button labelled "UPLOAD" is being pressed, a file explorer window opens. Through this window, the user navigates to the desired file. To select the file, either double click on the file or single click and press the "open" button on the bottom right.

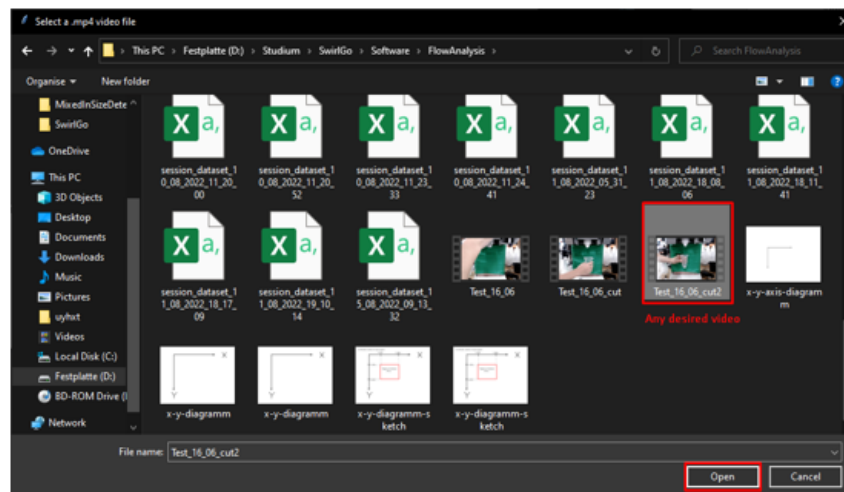


Figure 50: Newly opened file explorer window. Select a file and confirm to upload the video

The button calls the `uploadVideo()` function. This function takes the video file location, chosen via the `askopenfilename()` function. This function opens the file directory window. If the file path is not empty, the path is stored in the variable "chosenvideo". This variable is a global variable. It can be accessed by the analysis function, without being passed into as a parameter. The functions are all called via button pressed. The function calls, within the button declarations, do not take variables as inputs. This issue is fixed by declaring the desired parameters as global variables.

Quellcode 6: upload video function

```
1 def uploadVideo():
2     tempdir = filedialog.askopenfilename(parent=window, ←
        initialdir="/",title='Select a .mp4 video file')
3     if len(tempdir) > 0:
4         video_name = tempdir
5         global chosen_video
6         chosen_video = video_name
7         output.delete(0.0,END)
8         output.insert(END, "VIDEO UPLOADED:" + '\n' + chosen_video + '\n')
```

The logbook output text field, located on the bottom of the program window, is a means to communicate with the user. It displays information and feedback for the user. In this case, if the upload is successful, a message like this will appear:

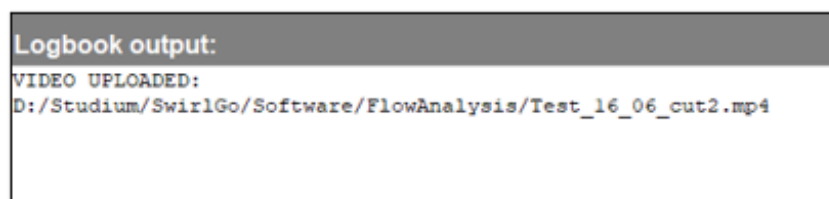


Figure 51: Logbook output upon successful upload

After uploading the video, the user is required to draw the region of interest within the video. The ROI is the region that will be important for the analysis. The rest of the video will be disregarded, since it will not affect the analysis results.

Press the "SUBMIT" button. A window will open, showing the video's first frame. To draw the ROI, left click on any spot of the window and drag and drop the cursor from the top left corner to the bottom right corner. The resulting ROI frame looks like this:

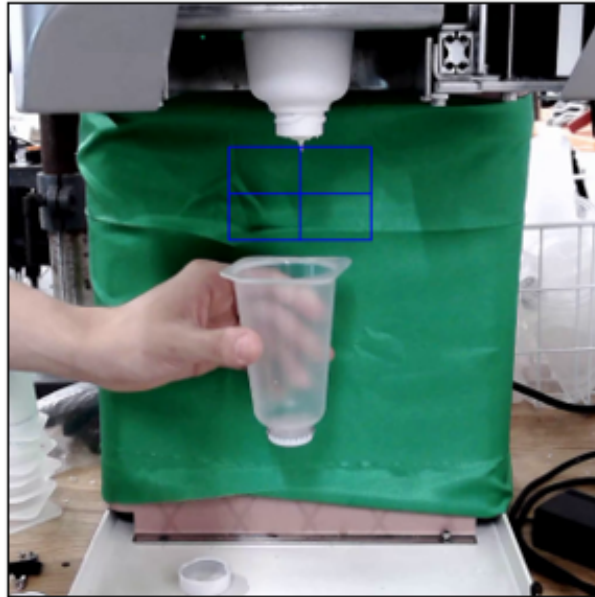


Figure 52: Result of drag and drop. Blue frame is the size of the ROI

The ROI parameters are the coordinates of the left, right, top and bottom border within the image. The parameters for the left and right border represent the maximum and minimum values on the x axis, while the parameters for the bottom and top border are the maximum and minimum values on the y axis.

The function makes use of error handling functions. These error handling functions contain several cases. The function first executes the case called "try". If an error is thrown within this case, the function will switch to an alternative path, such as the "except" case. This case is being executed instead, if the "try" case throws an exception error. These error handling functions can have several more cases. In this program, the functions only make use of the try and except cases. The use of such error handling functions greatly increases the program's robustness and prevents crashes.

The function will try to open the video file and display the first frame of the chosen video. The function, used to draw the ROI within the image is called "selectROI()". The dimensions are being passed as x and y values of the top left and bottom right corner. The coordinate values are passed to the border variables. The values are printed out in the output window. The ROIborders array is stored as a data file. These values will be loaded if an error occurs within the first case. These values are again being printed in the output text window.

 Quellcode 7: set roi borders function

```

1 def setROIborders():
2     global leftBorder
3     global rightBorder
4     global topBorder
5     global bottomBorder
6     leftBorder = 120
7     rightBorder = 170
8     topBorder = 120
9     bottomBorder = 170
10    ROI_borders = (0,0,0,0)
11
12    try:
13        ROI_borders = pickle.load(open("ROI_borders.dat", "rb"))
14    except:
15        leftBorder = 120
16        rightBorder = 170
17        topBorder = 120
18        bottomBorder = 170
19        ROI_borders = np.zeros(4)
20    try:
21        video = cv2.VideoCapture(chosen_video)
22        ret, frame = video.read()
23        if ret == True:
24            image = frame
25            region = cv2.selectROI(image)
26            leftBorder = int(region[0])
27            rightBorder = int(region[2]) + int(region[0])
28            topBorder = int(region[1])
29            bottomBorder = int(region[3]) + int(region[1])
30            ROI_borders = np.zeros(4)
31            ROI_borders[0] = leftBorder
32            ROI_borders[1] = rightBorder
33            ROI_borders[2] = topBorder
34            ROI_borders[3] = bottomBorder
35            output.delete(0.0,END)
36            output.insert(END, "Set ROI borders." + '\n')
37            output.insert(END, "left border = " + str(leftBorder) + '\n')
38            output.insert(END, "right border = " + str(rightBorder) + '\n')
39            output.insert(END, "top border = " + str(topBorder) + '\n')
40            output.insert(END, "bottom border = " + str(bottomBorder) + '\n')
41            pickle.dump(ROI_borders, open("ROI_borders.dat", "wb"))
42    except:
43        ROI_borders = pickle.load(open("ROI_borders.dat", "rb"))
44        ROI_borders[0] = leftBorder
45        ROI_borders[1] = rightBorder
46        ROI_borders[2] = topBorder
47        ROI_borders[3] = bottomBorder
48        output.delete(0.0,END)
49        output.insert(END, "Rolling with stored values." + '\n')

```

```

50     output.insert(END, "left border = " + str(leftBorder) + '\n')
51     output.insert(END, "right border = " + str(rightBorder) + '\n')
52     output.insert(END, "top border = " + str(topBorder) + '\n')
53     output.insert(END, "bottom border = " + str(bottomBorder) + '\n')

```

To confirm the selection, press enter and close the window. The border parameters are being displayed in the Logbook output window. The values represent the ROI border coordinate locations in pixels. A representation the coordinate system is displayed in figure 53.

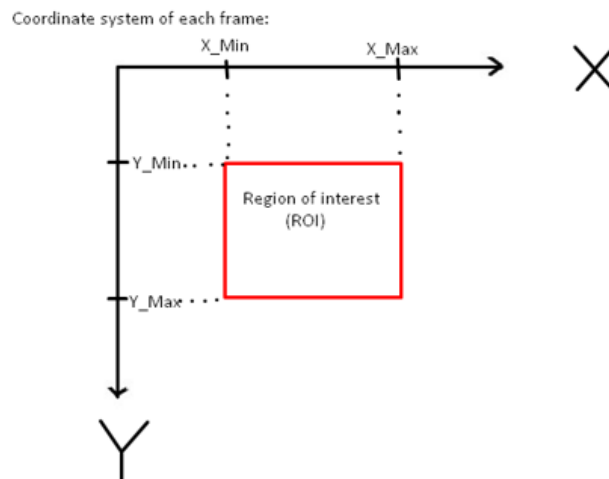


Figure 53: coordinate system of a picture with the ROI inside

To adjust the analysis function's sensitivity, adjust the value of the intensity value slider. The higher the value, the less sensitive the analysis program will be. The lower the value, the more sensitive it is. Confirm the value by pressing the "SUBMIT" button.

The function calculates the thresh hold values for the analysis function's canny edge detector. The canny detector uses these thresh holds to classify edge pixels into non, weak and strong edge pixels. Each pixel, after being convoluted with a filter, has a magnitude and direction. If the magnitude is lower than the lower thresh hold, it is not being classified as an edge pixel. If the magnitude is between both thresh hold values, it is classified as a weak edge pixel. If the magnitude is higher than both thresh holds, it is being classified as a strong edge pixel.

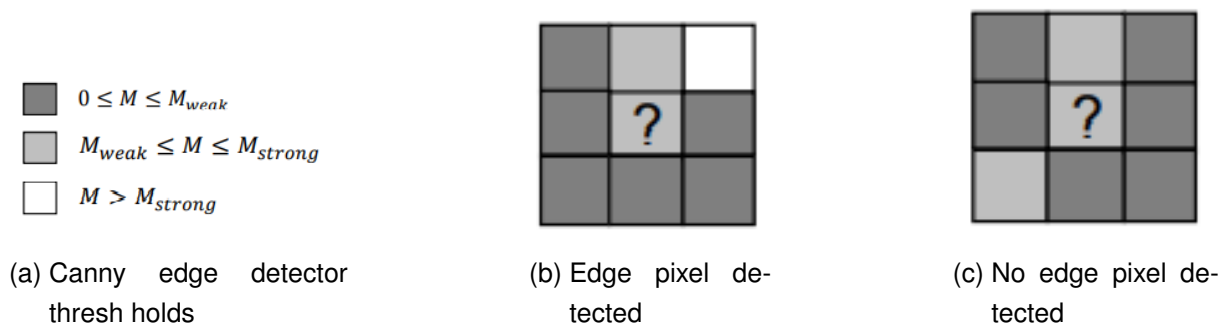


Figure 54: Canny edge detector thresh holds and use ??

The classification of edge pixels is important for the following processing steps of the canny edge detector. As a result, the edge detector marks all border pixels of each object within an image. This data is being used to calculate the size of the flow within the image.

The program tries to read the value, set in the slider, to calculate the lower and upper border thresh hold. The values are printed out in the output text window and the values stored within a data file. If the slider's value cannot be read, the most recent data file is being loaded and its values used instead.

Quellcode 8: set border function

```

1 def update_border_params():
2     global lowerBorder
3     global upperBorder
4
5     try:
6         borders = pickle.load(open("borders.dat", "rb"))
7     except:
8         lowerBorder = 150
9         upperBorder = 190
10    borders = np.zeros(2)
11
12    try:
13        lowerBorder = max(0, int(slider.get()) - 25)
14        upperBorder = max(int(slider.get()) + 25, 50)
15        borders[0] = lowerBorder
16        borders[1] = upperBorder
17        output.delete(0.0, END)
18        output.insert(END, "Set Canny Detector parameters." + '\n')
19        output.insert(END, "lower border = " + str(lowerBorder) + '\n')
20        output.insert(END, "upper border = " + str(upperBorder) + '\n')
21        pickle.dump(borders, open("borders.dat", "wb"))
22    except:
23        borders = pickle.load(open("borders.dat", "rb"))
24        lowerBorder = borders[0]
25        upperBorder = borders[1]
26        output.delete(0.0, END)
27        output.insert(END, "Rolling with stored values." + '\n')
28        output.insert(END, "lower border = " + str(lowerBorder) + '\n')
29        output.insert(END, "upper border = " + str(upperBorder) + '\n')
30
31    video = cv2.VideoCapture(chosen_video)
32
33    while(video.isOpened()):
34        ret, frame = video.read()
35        if ret == True:                                     #if you put in a ↵
36            different frame size, first look at the image size
37            image = frame
38            borders[0] = lowerBorder
39            borders[1] = upperBorder
40            output.delete(0.0, END)

```

```
40     output.insert(END, "Set canny parameters." + '\n')
41     output.insert(END, "lower border = " + str(lowerBorder) + '\n')
42     output.insert(END, "upper border = " + str(upperBorder) + '\n')
43     pickle.dump(borders, open("ROI_borders.dat", "wb"))
44     video.release()
```

Press the start button to continue with the analysis. A window will open, showing the video, overlaid with the result of the analysis on each frame. The analysis first finds out which pixels within the image belong to the stream. The borders of the flow are being marked in each frame. It gives a feedback to the user, whether the analysis is working as intended. If the borders follow the flow's shape, the analysis will yield valid results.

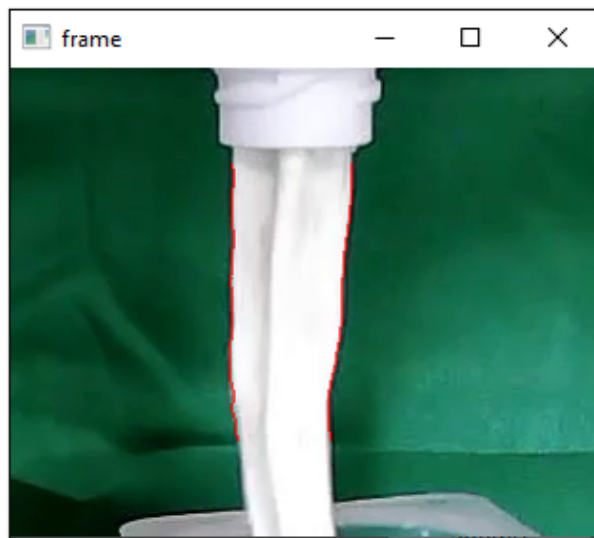


Figure 55: A frame, that has been analysed. The red pixels represent the flow's borders

The analysis function returns lists containing the average width, hue, intensity and saturation of the flow within each pixel. These values are plotted against the time axis represented by the list called time. To be able to use these variables in the post processing function, they are declared as global variables.

The function tries to copy the values of the global variables, that were initialised in the previously mentioned functions. If any of the values throws an exception error, the function loads the values from the most recent data files.

The video is then being opened and read. The analysis takes place within the while loop that is executed while the video is opened. As soon as the video is being opened, it is being read frame by frame. The flow's analysis is being processed over each frame. The data of all frames combined shows the flow's width and colour development over time.

All frames are cropped to the size of the ROI. The ROI is buffered around its edges showing a part of the frame that is outside of the region. The cropping improves runtime for later stages in the program. Having a small buffer around the ROI gives the user a context what is happening outside of the ROI while not affecting the runtime and analysis.

As soon as the video is being opened, it is being read frame by frame. For each frame, the

canny edge detector is run over the picture. The result is a frame, showing the contours of all objects within the image. The canny edge detector is a function that is being used to detect borders between objects within an image. At the edge of an object, there is usually a difference in colour values, that can be detected via edge detection through calculating the image's colour gradient.

The canny edge detector works in five steps. First the picture is being low pass filtered via a Gauss filter. The Gauss filter slightly blurs the image. This filters out any high frequency white noise.

After blurring the picture, calculate the intensity gradient (amplitude and phase). The non maximum suppression suppresses any pixels, that are not the maximum along an edge.

The resulting edge pixels are classified in three categories. Pixels are classified in not edge pixels, weak and strong edge pixels. Strong edge pixels are guaranteed to stay edge pixels. Weak edge pixels are looked over again in the hysteresis tracing.

If a strong edge pixel is within a defined range of a strong edge pixel, it is perceived as a strong edge pixel. If no strong edge pixel is within the range, the pixel will instead be seen as a non edge pixel [14].

The workwise of the canny edge detector is divided into five phases. These five phases all have their own purpose for the final goal. The canny edge detector consists of these five functions:

- Gauss Filtering
- Calculation of intensity gradient
- Non-Maximum suppression
- Double thresh hold
- Hysteresis tracing

The canny edge detector is an edge detector with a low amount of noise and binary output. This binary output makes it easier to use for further steps in the analysis. The edge detector ideally finds the edge of all contours within the image and gives each detected edge a width of one pixel. The high accuracy of the canny edge detector makes it an ideal tool for the flow width analysis. The flow's contour is being detected by the edge detector and then used for the following analysis steps [14].

Additionally, the frame is converted from the RGB colour space to the HSV colour space. This is used for the colour analysis part of the flow analysis.



Figure 56: RGB and HSV colour space representations

The RGB space as shown in figure 56a illustrates the colour space. The RGB colour space is a three dimensional, cubic space. The HSV colour space resembles the shape of a cylinder as shown in the illustration in figure 56b. The colour of a pixel within the RGB colour space is defined by its three colour channels red, blue and green. Each of these values influences the resulting colour's hue, its saturation and brightness. This poses an issue for later analysis. The RGB colour representation has higher influxes of noise than the HSV representation. Variables within the RGB space are more prone to be disturbed by external influx such as light intensity and colour changes.

The HSV colour space separates the most important variables from each other. The hue, saturation and brightness are all independent from each other. This makes for a more robust representation. For example, a change in light brightness will have its most effect on the brightness of a pixel but not so much on the hue and saturation. While in the RGB space, this would have affected all variables at once, it is only significantly noticeable in one of the HSV colour space variables. A more robust colour representation allows a better evaluation of the flow's colour and a lower amount of noise within the resulting samples.

For each valid frame containing information, the function iterates through all rows and columns within the ROI. For each row, the function looks for the furthest left and furthest right edge pixel. These pixels are later being referred to as minimum for the furthest left and maximum for the furthest right edge pixel. This name is chosen because of the pixel's index within the picture.

All rows up to a point, called the cutoff point, are being included into the coming calculations. If at one point the the width is zero, the row is being marked as the stream's cut of point. In this row, the minimum is the same as the maximum. This is the point, where the stream has abruptly ended. Rows underneath this row will be ignored in the stream's width and colour calculation of this frame. For all rows above the cutoff point, the distance between the stream's edge pixels is calculated and added to the normed distance list. The hue, intensity and saturation of all pixels between the maximum and minimum is stored in its respective lists.

The average width of all valid rows, within each frame is being calculated. The flow is never flowing completely vertically. This is why the flow's average angle has to be considered in the calculation, too. If the flow is tilted, the program, without knowing the flow's angle, will see the flows width as wider than it actually is. To calculate the angle, take the x and y coordinates of

all middle pixels within the analysed rows. A polyfit function is being called, calculating a linear approximation of these points. This function from the numpy library calculates the slope and offset of the linear function, that approximates the points with the minimal error.

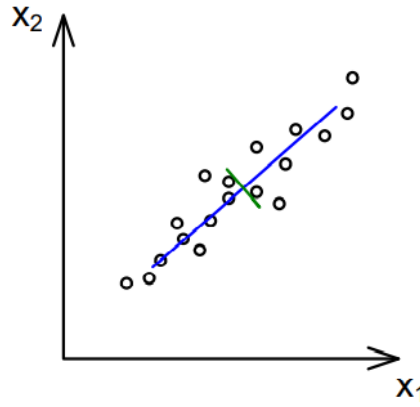


Figure 57: Example of pixel cluster polyfitting [16]

The magnitude of the resulting line is used to calculate the flow's angle. The average width is multiplied with the cosine of this angle to adjust the flow's orientation.

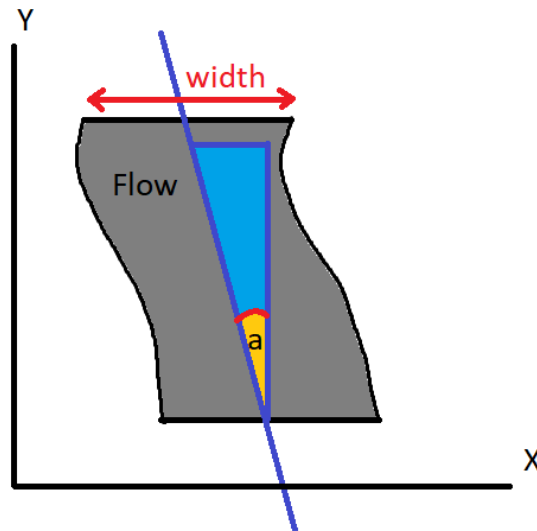


Figure 58: Angle of the flow

If the list for the hue is not empty, calculate the average of the hue, saturation and value of the stream within the frame. Append the average of this frame to the list of all colour variables over all frames. These lists will be used for the final plots.

To scale the time axis from number of frames to seconds, divide the frame numbers through the frames per second. In this case the video is played with 30 frames per second. The width over time is the first plot, that will be returned. Every time the flow has been cut off, a vertical,

red, dotted line is being plotted on the cut off time. This gives the user further feedback to make a better evaluation. The red line will be visible in all other plots as well.

After the width over time graph is closed, the program plots the hue over the course of time. The most important data such as the hue average, its maximum and minimum as well as its variance are being printed in the output window. The values are being calculated before the plot is shown. The maximum and minimum are marked with horizontal, dotted lines within the graph. The wider these lines are apart, the more the flow's colour has changed over time. Ideally, the gap between these values should be as small as possible. If the gap is zero, it means that the flow's colour has not changed over the course of time.

The same process is being repeated for the saturation and intensity. The same quality measurements are being applied to these metrics.

As soon as all plots are closed, the gathered data is being passed into a .csv file. The data can be accessed via excel and used for further analysis if needed.

The post processing part is used to clear the data of any white noise. This part takes four additional parameters as inputs. These inputs are used to smooth the graphs and classify the colours of the flow. If the flow has changed its colour the program will be able to find, when the colour change took place.

The data is being low pass filtered by a moving average filter. The first parameter determines the moving average filter's length. The more samples the moving average filter takes, the more the curves will be smoothed. This only works up to a certain extend. If the moving average filter takes too many samples, it will start to distort the graph's shape.

This can be seen the most for the graphs of the colour values. The width over time plot has not been affected by the low pass filter too much in that regard. The following solution has only been applied for the metrics of colour.

A solution had to be found, where white noise will be filtered out, without changing the original graph's general shape. A viable solution is to approximate the curve. In this case, the polyfit function has been chosen for this task. The polyfit function's rank is set at 40. The parameters are used to calculate the values over time of the approximated curve. The approximated curve has the ability to filter out the noise while retaining the original curve's shape.

Because of the missing white noise, the gap between maximum and minimum for the colour values is usually narrower than of the original curve. Usually, spikes, caused by white noise will falsely make the gap look wider than it actually would be without. Without the white noise, more accurate assumptions about the flow's quality can be made. The data of the low pass filtered and the approximated curves are being printed in the output field. Both curves are being plotted in the same graph. The approximated curve and its maximum and minimum indicator lines are being shown in red, while the low pass filtered graph and its outliers are being shown in blue. The red curve is not affected by white noise. This makes the data, gathered from the approximated cure more useful.

Additionally, the function classifies the flow's colours in clusters. The classifier uses DBSCAN. The user is asked to adjust the radius used inside the process of the DBSCAN. The HSV colour values are sorted and then passed through the scan. The function iterates through each sorted colour variable list and looks for the point at which the differences between two

values exceeds the radius. At these points, a marker is set within the graph. Colour changes are indicated via a green, dotted, vertical line at the point of colour change. This is being done for the hue, saturation and value graphs.

The classification automates the process of finding the time, at which the colour changed. Colour changes can occur through an inhomogeneity of mixed ins within the substance. This reflects on the SwirlGO machine's performance in the mixing process.

5.3 mixed-in size-detector

This program calculates the size of all mixed ins that will be used for a test sample. The program takes an image as input. The image is a picture of all used mixed ins, that will be used for the sample.



Figure 59: Example picture used as input

The main program is constructed in a similar fashion to the flow analysis program. The first part contains all functions, used to initialise the variables, that are needed as variables for the program's analysis part. Just like the previous program, the functions, that are used to initialise these variables, are accessed by the buttons inside the UI.

Quellcode 9: functions within the size detector program

```
1 def update_canny_params()
2
3 def update_reference_dimensions()
4
5 def click_upload()
6
7 def close_window()
8
9 def _rect_inter_inner(x1,x2)
10
11 def _rectangle_intersection_(x1,y1,x2,y2)
12
13 def intersection(x1,y1,x2,y2)
14
15 def scaling_programm_part()
16
17 def find_object_dimensions()
```

The second part consists of the analysis functions. These functions execute the program's main part. The functions associated with this part are the functions, calculating the intersections of curves and the scaling function.

The part of the program, that connects all functions together into a functioning unit, is the UI. It is constructed in the final part of the code.

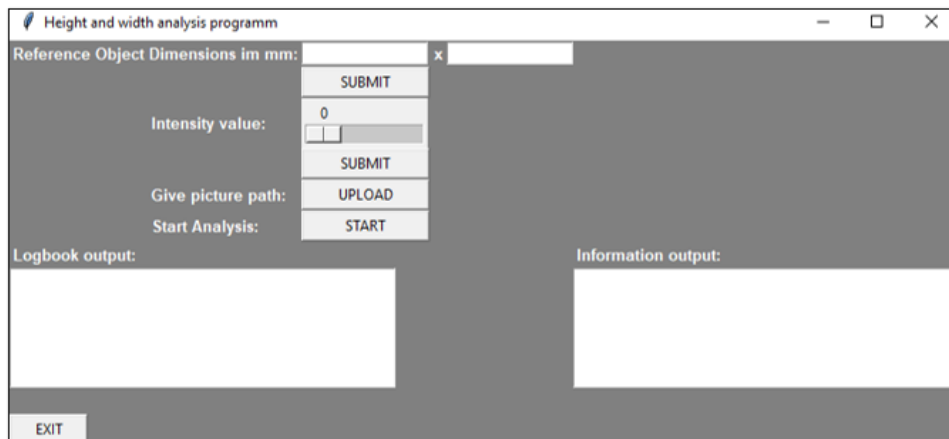


Figure 60: UI of the mixed in size detector

The UI consists of the same elements as the previous program, with the addition of the textentry field. The textentry field enables to communicate with the UI by typing text in it. In this case, the user is supposed to type in a floating point number. The text fields are used to type in the height and width in mm of the reference object. The reference object is being used to calibrate the camera. Since the height and width of the reference object are known, it makes it easy for the program to calculate the numbers of pixels that represent one millimetre in the real world.

The submit button calls the functions to update the reference object dimensions.

Quellcode 10: function to update reference object dimensions

```

1 def update_reference_dimensions():
2     global width_in_mm
3     global height_in_mm
4     global scalings
5     width_in_mm = 24
6     height_in_mm = 18.6
7     scalings = np.zeros(2)
8
9     if textentry_reference_dimensions_x.get() != "" and ↵
        textentry_reference_dimensions_y.get() != "":
10         try:
11             width_in_mm = float(textentry_reference_dimensions_x.get())
12             height_in_mm = float(textentry_reference_dimensions_y.get())
13             scalings[0] = width_in_mm
14             scalings[1] = height_in_mm

```

```
15         output.insert(END, "Adjusted reference object dimensions." + ↵
        '\n')
16         output.insert(END, "x = " + str(width_in_mm) + '\n')
17         output.insert(END, "y = " + str(height_in_mm) + '\n')
18         pickle.dump(scalings, open("scalings.dat", "wb"))
19     except:
20         scalings = pickle.load(open("scalings.dat", "rb"))
21         width_in_mm = scalings[0]
22         height_in_mm = scalings[1]
23         output.insert(END, "Rolling with stored values." + '\n')
24         output.insert(END, "x = " + str(width_in_mm) + '\n')
25         output.insert(END, "y = " + str(height_in_mm) + '\n')
```

Similar to the flow analysis program, the variables, that will be accessed by the analysis program, are declared as global variables. In this case, the width and height of the reference object are being passed to the analysis function.

If the text entry fields are kept empty, the width and height are kept at the initial values of 20 mm and 18.6mm. If the textentry fields have at least one character typed in each, the function reads the textentry fields' contents. The contents will also be stored in the array called scalings. The data is then stored in a data file. The scalings are printed out in the logbook output window.

If an error occurs, the function will instead load the most recent data file, that contains the variables, and use them. The values are also being printed out, along with the message, that it is the stored values, that have been used instead.

The analysis function's functionality is primarily based on the canny edge detector. The slider to adjust the intensity value is being used to adjust the thresh holds of the canny edge detector. The higher the slider's value, the higher the thresh holds of the canny edge detector are. The higher the thresh holds, the more extreme the contrast between pixels has to be, to be classified as an edge. The opposite applies for lowering the slider's value. The value is set by pressing the submit button underneath the slider. The button calls the function to update the canny edge detector parameters.

Quellcode 11: function to update canny edge detector thresh holds

```
1 def update_canny_params():
2     global lowerBorder
3     global upperBorder
4
5     borders = np.zeros(2)
6     lowerBorder = max(0, int(slider.get()) - 25)
7     upperBorder = max(int(slider.get()) + 25, 50)
8     borders[0] = lowerBorder
9     borders[1] = upperBorder
10    output.insert(END, "Set Canny Detector parameters." + '\n')
11    output.insert(END, "lower border = " + str(lowerBorder) + '\n')
12    output.insert(END, "upper border = " + str(upperBorder) + '\n')
13    pickle.dump(borders, open("borders.dat", "wb"))
```

The function works the same way as the previous function. The two parameters, that need to be set are the upper and lower thresh hold of the canny edge detector. The canny edge detector's functionality has been described in the previous subtask.

The user then has to chose the picture file within the computer file system. Upon pressing the upload button the upload function is being called.

Quellcode 12: function to upload the picture to be analysed

```
1 def click_upload():
2     output.delete(0.0,END)
3     global image
4     tempdir = filedialog.askopenfilename(parent=window, ↵
        initialdir="/",title='Select a .mp4 video file')
5     if len(tempdir) > 0:
6         image = tempdir
7         output.insert(END, "UPLOADED IMAGE:" + '\n')
8         output.insert(END, image + '\n')
```

The function opens a file directory window, through which the user has to navigate to give the program the exact location of the desired file. If the path is not empty, the path is being copied to the image variable. The path is printed out in the logbook textfield as feedback for the user.

If all previous steps have worked out up to this point, the user can now proceed with the analysis. Pressing the start button executes the function to find all objects' dimensions within the image.

Quellcode 13: analysis function

```
1 def find_object_dimensions():
2     scaling_programm_part()
3     output.insert(END, "Processed all object hulls." + '\n')
```

This function calls another function, in which the calculations are being made. The function first loads the previously initialised variables. If an error arises, a message is printed out in the output window and the function is exited.

The picture is being processed with the canny edge detector. The result is a black and white image containing the contours of all objects within the image. The resulting picture is then dilated and afterwards eroded with a 5x5 filter.

Take the following 3x3 filter matrix. This matrix will be used to explain the way morphological operators such as dilation and erosion work. This matrix will be applied as a filter over other matrices. The matrices represent black and white images. The pixels with value zero represent black pixels. Pixels with value 255 represent white pixels. White pixels represent edge pixels in the image.

$$\begin{bmatrix} 255 & 255 & 255 \\ 255 & 255 & 255 \\ 255 & 255 & 255 \end{bmatrix}$$

Figure 61: 3x3 Filter for dilation and erosion [14]

The matrix shown below is an example of an image. The filter matrix will be used to apply a dilation over the image. A dilation is sometimes also being referred to as a "logical or operator". Every time at least one white pixel is within a part of the picture, that is inside the filter's window size, the resulting pixel on this position will be white. Otherwise, the resulting pixel will be black.

$$\begin{pmatrix} 0 & 0 & 0 & 255 & 255 & 255 \\ 0 & 0 & 0 & 255 & 255 & 255 \\ 0 & 0 & 0 & 255 & 255 & 255 \\ 0 & 0 & 0 & 255 & 255 & 255 \end{pmatrix}$$

Figure 62: Example matrix before dilation [14]

The convolution uses no padding. This means that the 4x6 matrix will result in a 2x4 matrix after the convolution with the 3x3 filter. The result shows, that only the furthest left column consists of black pixels. The portion that is being covered by the filter has no white pixels in this area, which results in a black pixel.

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 255 & 255 & 255 & 0 \\ 0 & 0 & 255 & 255 & 255 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Figure 63: Dilated matrix [14]

Erosion is sometimes being referred as a "logical and operator". It can be seen as the opposite of the dilation operation. If all pixels within the filter covered area within the image are white, the resulting pixel will also be white. If at least one pixel within the covered area is black, the resulting pixel will be black.

$$\begin{bmatrix} 0. & 0. & 0. & 255. & 255. & 255. & 255. & 255. \\ 0. & 0. & 0. & 255. & 255. & 255. & 255. & 255. \\ 0. & 255. & 255. & 255. & 255. & 255. & 255. & 255. \\ 0. & 255. & 255. & 255. & 255. & 255. & 255. & 255. \\ 0. & 0. & 0. & 255. & 255. & 255. & 255. & 255. \\ 0. & 0. & 0. & 255. & 255. & 255. & 255. & 255. \end{bmatrix},$$

Figure 64: Example matrix before erosion [14]

The resulting matrix shows the results of the erosion. The three furthest left columns are being interpreted as completely black; a result of the black pixels. Again, the resulting picture is smaller than the original picture, because no padding has been applied.

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 255 & 255 & 255 & 0 \\ 0 & 0 & 0 & 0 & 255 & 255 & 255 & 0 \\ 0 & 0 & 0 & 0 & 255 & 255 & 255 & 0 \\ 0 & 0 & 0 & 0 & 255 & 255 & 255 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Figure 65: Eroded matrix [14]

Applying dilation on the picture first and then eroding the first result is also being called the "closing" operation. This morphological operation is useful for bridging smaller gaps and closing holes within pixel clusters. In this case, it is being used to clear white noise from the resulting image of the canny edge operation.

On the eroded picture, the function calls the function "cv2.findContours()". This function outputs the coordinates of all points within each convex hull and its place in the hierarchy. The hierarchy holds the information for each hull's parents. This function finds the convex hulls of all objects within an image. The resulting convex hulls will be interpreted as each object's shape. Each convex hull is sorted within a certain hierarchy. The hull, that surrounds an other hull is called a parent. The hulls that are being taken into account for the analysis are the hulls, that have no surrounding hull around them / no parent hull. The picture below shows an example of several convex hulls. Hull A is the parent of hulls B and C because it completely surrounds them. Hull A on the other hand does not have a parent.

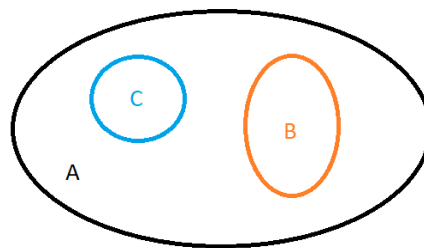


Figure 66: Example of hulls within their parent hull

A convex hull defines a hull that has no interior angles greater than 180 degrees. A convex hull can be used to make an approximation of an object within an image. An object within an image can be represented as a cluster of pixels. The picture shown below, gives an example of how a convex hull, surrounding a set of points within a cluster looks like.

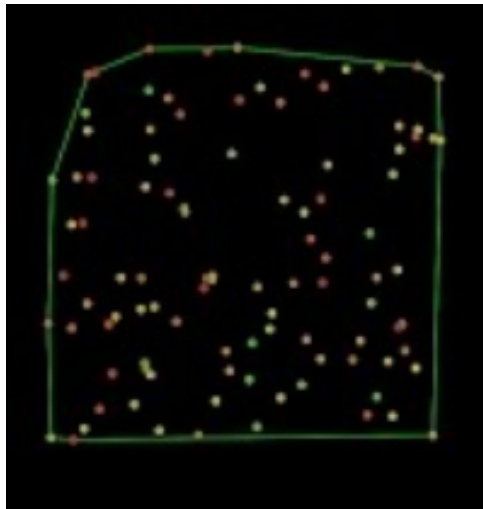


Figure 67: Example of a convex hull[15]

The contours are drawn in the image. The image is then being shown as feedback for the user. The contours are coloured in green.

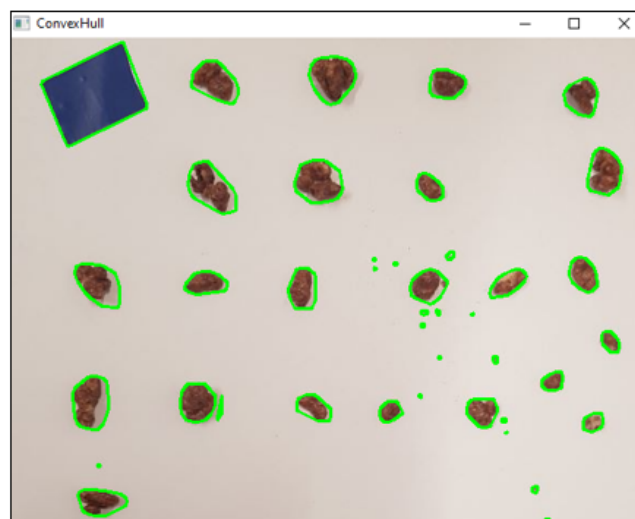


Figure 68: drawn contours around all detected objects

As soon as the user closes the window, the program starts the calculation of each object's height and width. To do so, each object's main axis has to be found. The distance between intersections of the main axis with the object's hull is the object's width. The normal line, crossing the main axis in the object's middle, is used to calculate the object's height. The distance between the intersections of the normal line and the object's hull is the object's height.

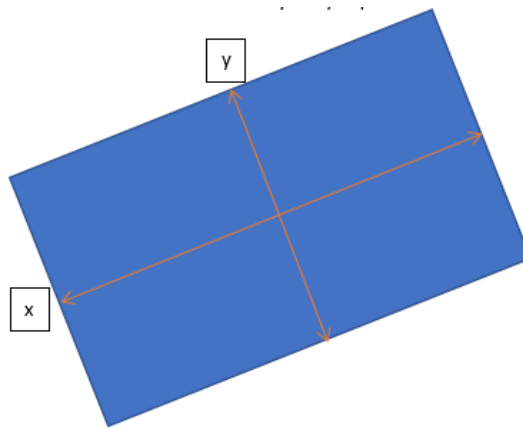


Figure 69: height (y) and width (x) of an object

The function iterates through all convex hulls within the image. If the hull is a hull without a parent, it is being considered in the following analysis part.

The hull's coordinates are being copied and the coordinate of the object's center calculated. Additionally, the minimum and maximum values of the x and y coordinates are being found. The object's coordinates are used in the polyfit function. The polyfit function is used to find the object's main axis parameters. These parameters are used to calculate the points of the main axis. The x and y coordinates of the intersections between the main axis with the object's coordinates are being calculated with the intersection function.

The intersection function calculates the locations of all intersections of two curves with each other. The intersection function makes use of the rectangle intersection function. The rectangle intersection function is used to define a region of interest in which the intersections can be detected. The region has a rectangular shape and has the dimensions of the shared area of both rectangular regions of interest. As seen in figure 70, the resulting area is the yellow area. The green and blue area are the ranges of both curves within the graph.

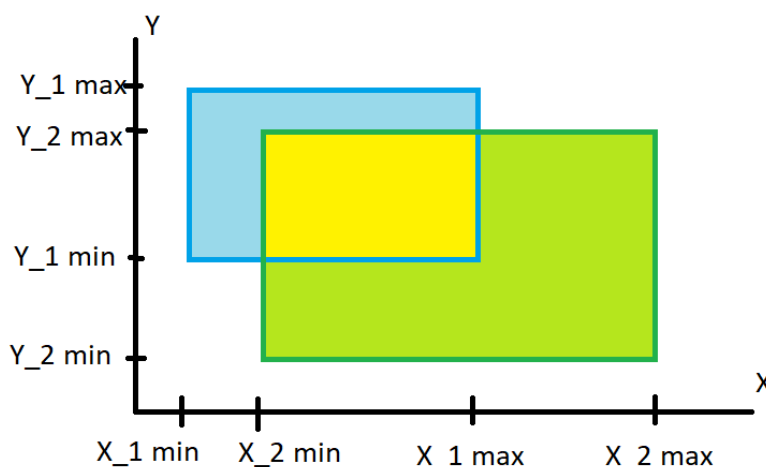


Figure 70: Resulting ROI in which the intersections are being calculated

The borders of the ROI are being calculated via the maximum and minimum x and y values of each function. This is possible for all functions, since in programming, curves are usually only calculated within a certain range of x and / or y values.

Continuing with the intersection function, the gradient for x and y values of the curve are being calculated and passed into the matrix AA. The x and y values for each point on the curves are passed to BB. The values for the x and y coordinates of all intersections are being calculated with the numpy function "np.linalg.solve()". The coordinates are stored in the array xy0, which is returned as the function output.

The functions have been coded by user John Paul Qiang Chen and posted on a thread in stackoverflow [23].

If there are at least two intersections with the main axis and the object's coordinates, the main axis' normal line is being calculated. The intersections of the normal axis and the object's hull are being calculated after that. Since the hull is convex, the number of intersections of these lines with the hull will always be two. There are certain cases where the program only detects one intersection. Since the data, that is yielded from these hulls is not able to be used in the later stages of the program, the value of the width and height are being set to zero and later being ignored by the program.

$$x_{middle} = \frac{x_{intersection,max} + x_{intersection,min}}{2} \quad (8)$$

$$y_{middle} = \frac{y_{intersection,max} + y_{intersection,min}}{2} \quad (9)$$

$$m_{normal} = \frac{-1}{m} \quad (10)$$

$$b_{normal} = y_{middle} - x_{middle} * m_{normal} \quad (11)$$

If the normal axis has at least two intersections with the object's coordinates, the object's height and width are being calculated. If there are less than two intersections, the height and width are being calculated using the coordinates of the maximum and minimum x and y values of the object.

Each analysed hull is being marked in the picture, by changing each analysed object's hull's colour within the image. The largest object will be marked in a different colour than all other objects. This helps the user also find out whether the program detected the correct object as the reference object. The resulting picture will be shown to the user as a feedback on how well the analysis went. The ideal analysis should have assigned the correct object as the reference object and calculated the height and width of every object within the image.



Figure 71: input picture with reference object and analysed objects marked

Using these measurements, the height and width of every object is thus recalculated from pixels to millimetres. If the height and width of an object is not been set to zero, it is re-scaled in millimetres and considered in the final plot. The objects that have a height or width equal to zero, are being ignored and not shown in the final output. These values are shown in the two resulting plots. Other useful information such as the average width and height are being printed in the information output textfield.

To close the program, press the "EXIT" button. The button calls the "close-window()" function. This function saves the data of the reference object before closing the main window and shutting down the program. The data will be available for the next session.

Quellcode 14: exiting the program

```
1 def close_window():
2     pickle.dump(scalings, open("scalings.dat", "wb"))
3     window.destroy()
4     exit()
```

6 Project 5: construction of stepper motor controller

This project is used to improve the lifetime of the motor inside the SwirlGO machine. The parameter that is being optimised is the motor's rpm. The parameter that puts a lot of stress on the motor is the acceleration of the rpm. If there are jumps in the acceleration, it puts stress on the motor. The goal of this task is to eliminate these jumps. To do so, a controller has to be implemented, that will smooth out the rpm curve. The difference between the goal and the current state can be seen in figure 72. The current curve has jumps in acceleration, while the goal curve's derivative is continuous.

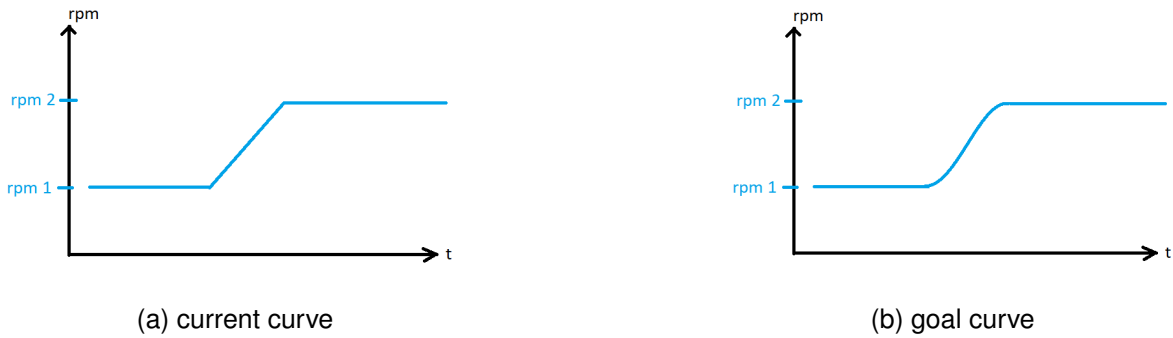


Figure 72: comparison between current curve and goal curve

The stepper motor has its own controller. The motor-controller system is stable. The system adjusts the values so quickly, that its step response is almost as quick as the sample frequency of the microcontroller. This is why the step response of the motor has not been measured in this project. The desired controller has the form of a PT2 element. The damping constant has been set to a value higher than one, to suppress the up and down swings of the PT2 element.

$$G(s) = \frac{k}{T^2 s^2 + 2DTs + 1} \quad (12)$$

$$D > 1$$

$$k \in \mathbb{R}$$

The controller is being discretized by substitution of the laplacian variable s with the tustin approximation. The transfer function is now located in the z -value space. The microcontroller's sample period time has been observed to be around 0.01 seconds in average. The deviation from this value have been observed to be minimal. Since the time continuous controller has just been approximated, the period time T of the controller has to be greater than four times of the sample period time.

$$s \rightarrow \frac{2}{T} \frac{z - 1}{z + 1} \quad (13)$$

$$T_s = 0.01s$$

$$T > 4 * T_i$$

The resulting discrete transfer function looks like this:

$$G(z) = \frac{r_2 z^2 + r_3 z + r_4}{z^2 + r_1 z + r_0} \quad (14)$$

$$T_s = 0.01s$$

$$T = 0.2$$

$$k = 1$$

$$r_0 = -0.9026$$

$$r_1 = 1.9$$

$$r_2 = 0.0005942$$

$$r_3 = 0.001188$$

$$r_4 = 0.0005942$$

For a period time T of 0.2 seconds, the parameters r_0 to r_4 will have the values shown above. The transfer function is being transformed into a discrete differential equation. The differential equation looks like this:

$$y_k = -r_0 y_{k-2} - r_1 y_{k-1} + r_2 x_k + r_3 x_{k-1} + r_4 x_{k-2} \quad (15)$$

The function is not stationary stable. Fortunately, the factor between the stationary end value of the controller's step response and the goal value is constant. The output variable has to be scaled by this factor, to achieve the stationary accuracy.

$$y_{output,k} = m * y_k \quad (16)$$

$$m \in \mathbb{R}$$

$$m = \frac{y_{goal}}{y_{stationary,controller}}$$

The controller's result can be seen in figure 73.

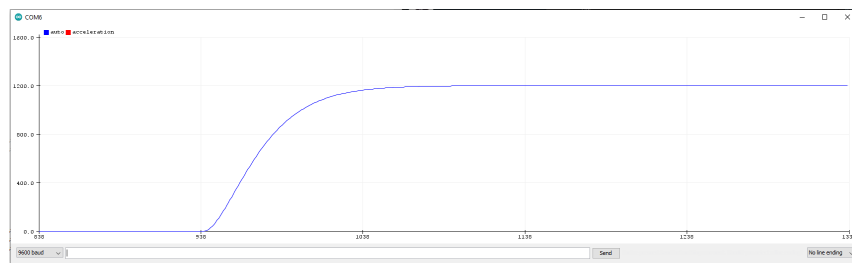


Figure 73: resulting rpm curve after controller implementation

7 Conclusion

I was given tasks that all together reflect most parts of the electrical and information scientific engineering profession. My tasks were primarily focused in the development of hardware and software of embedded systems of mechatronical machines and the development of testing methods. I was able to experience the different stages of a development project and work in a team consisting of people coming from different areas of expertise.

The development tasks gave me the opportunity to put my theoretical knowledge from lectures into practice. It improved my understanding of the theory and gave me more confidence in my own skills. Having to come up with my own ideas, without having a clear laid out path on how to achieve a desired goal, was a fun challenge.

Taking responsibility over several projects, that either be shipped to customers or used within the company, gave me the feedback that my work mattered. This gave me the motivation to give my best and take on the challenges that came along with the development.

The development tasks showed me the importance of regular, organised testing. To produce a robust and reliable product, you have to repeatedly test the product and analyse any possible error source and find solutions for them. Seeing the importance of iterations in all parts of the development process also made me aware about how crucial a good documentation and development process structure is. Getting closer to the final goal with every iteration also gave a feeling of the project's current progress. A clear and precise analysis of any problem is the basis for good quality development. Thus, learning how to conduct proper analysis and research of the problems also improved my developing skills.

Getting clear specifications for each product, such as main goal parametrics, deadline and budget, also gave me an insight into the economics of the engineering profession. Before entering the company, I was not aware as much about the main goal of a project to return a net profit for the company as a whole. This became important during the company's weekly meeting, where we discussed all projects' current progresses and assigned tasks for the following days.

Each stakeholder had different interest in the project. For example, the company's leadership is more interested in the project's cost, progress and estimated earnings. The engineering team on the other hand is more focused on the workwise of the machine as well as the understanding and current state of the project and its resulting product. Mediating with these different stakeholders helped me further develop my conversational skills. This also proved to be one of the most important skills to have in group projects.

In total, the internship taught me a lot of crucial skills and showed me a lot of parts about the engineering occupation, that I did not take into account before.

8 Future

Before starting my internship in Advantir Innovations, I was unsure which area of expertise within the electrical and information scientific engineering occupation interested me the most. During my university studies, I found my interest in the field of robotics. The robotics field includes almost all professions from artificial intelligence programming to electrical engineering and mechanics down to material sciences. I focused myself primarily on the mechatronical part and the artificial intelligence part of robotics. However, I did not have much practical experience when it came to developing my own projects. This made me question which part of robotics would interest me the most. On one hand, I did not want to do just programming. On the other hand, I did not imagine myself just working on electrical and mechanical hardware. I was looking for something that would combine both of the fields I was interested in.

The internship offered me the opportunity to find this balance between both fields in the development of embedded systems for mechatronics and their test methods. I now have a clearer understanding of what to expect in my future career and found the direction, I want to take my studies to. It gives me more confidence in my decisions and an idea of what to expect in my future.

9 List of Abbreviations

LED light-emitting diodes

DIN digital input

DOUT digital output

RPM rotations per minute

PWM pulse width modulation

DC direct current

AC alternating current

GPIO general purpose input/output

RGB red green blue

HSV hue saturation value

V Volt

A Ampere

N Newton

m metre

ROI region of interest

CV computer vision

UI user interface

PCB printed circuit board

AIR auger insertion rig

BOM bill of materials

DBSCAN density-based spatial clustering of applications with noise

List of Figures

1	Swirl.Go logo [2]	4
2	Location of Advantir Innovations [3]	4
3	Circuit diagram of the interface in LTspice	7
4	Capsule lighting circuit diagram in LTspice	7
5	Low pass filter constructed from resistor and capacitor	7
6	Sketch of the LED strip model, used in the program	9
7	Sketch of the fading matrix dimensions	10
8	Fading effect matrix iteratively passing through the LED strip and adjusting the colour values	10
9	Problem shown with the LED strip	12
10	Diffused cone light passing through gap	12
11	Diffusion of light in relation to surface roughness [17]	13
12	Influx of light coming from neighbouring LED	13
13	Influx of light limited by dividing wall between LED	13
14	top view of the LED strip's light cover / guide	14
15	Smooth lighting from LED strip	14
16	Development process stages for the AIR	15
17	Problem analysis for AIR development	16
18	Picture and Sketch of the AIR's first iteration	16
19	Comparison of step response of a PT2 element to the response of the motor	17
20	4 Quadrant Chopper current flow for positive and negative output current. The Output current is the current that flows through the induction in the middle, which represents one of the motor coils [6]	18
21	Wiring of the controller, motor and board to each other [7]	18
22	Nema 17 Image [18]	19
23	Output current of the Chopper depending on the amount of microsteps [6]	19
24	Continuous step response characteristics of the stepper motor at different speeds [4]	20
25	Resulting forces and moments, acting on the insertion rig (side view)	25
26	Reinforcement on the back of the machine	26
27	System chart of the auger insertion rig	27
28	Picture of the first finished prototype with UI	28
29	Cable management of breadboard prototype	29
30	Final result for the PCB	30
31	File organisation in Altium designer	30
32	First PCB milestone	31
33	Interface of the trace width calculator	32
34	PCB traces of top and bottom layer. Large areas of blue and red are areas of copper, connected to the ground	33
35	use and advantages of inserted planes connected to ground	34

36	2 Pin connector and header	34
37	female header, used to hold the board [20]	35
38	female header, used for the digital connections of the motor drivers [19]	35
39	Capacitor and Resistor used in the PCB	35
40	two versions of the second generation	36
41	solders of the two versions of the second generation	37
42	Suggested motor-driver-sets	39
43	example of a rotary torque sensor [12]	40
44	Picture of a good sample	41
45	Sketch of the cup spinner machine	42
46	TMC2209 V3.0 stepper motor driver	42
47	Comparison of ideal run with test results	43
48	Video recording stand	45
49	Flow Analysis Program Interface	45
50	Newly opened file explorer window. Select a file and confirm to upload the video	47
51	Logbook output upon successful upload	47
52	Result of drag and drop. Blue frame is the size of the ROI	48
53	coordinate system of a picture with the ROI inside	50
54	Canny edge detector thresh holds and use ??	50
55	A frame, that has been analysed. The red pixels represent the flow's borders . .	52
56	RGB and HSV colour space representations	54
57	Example of pixel cluster polyfitting [16]	55
58	Angle of the flow	55
59	Example picture used as input	58
60	UI of the mixed in size detector	59
61	3x3 Filter for dilation and erosion [14]	62
62	Example matrix before dilation [14]	62
63	Dilated matrix [14]	62
64	Example matrix before erosion [14]	62
65	Eroded matrix [14]	63
66	Example of hulls within their parent hull	63
67	Example of a convex hull[15]	64
68	drawn contours around all detected objects	64
69	height (y) and width (x) of an object	65
70	Resulting ROI in which the intersections are being calculated	65
71	input picture with reference object and analysed objects marked	67
72	comparison between current curve and goal curve	68
73	resulting rpm curve after controller implementation	70
74	Data sheet of the currently used motor [8]	78

Literature

- [1] "v4 Swirl.GO: Colour, Material, Finish"
Power Point presentation from November 2021, Updated in February 2022
- [2] <https://www.facebook.com/swirlgosg> (1.5.22)
- [3] <https://www.google.com.sg/maps/place/Advantir+Innovations>
(1.5.22)
- [4] <https://blog.orientalmotor.com/tips-to-minimize-stepper-motor-vibration>(5.5.22)
- [5] Vorlesung Systemdynamik und Regelungstechnik
Prof. Dr. Ing. Sören Hohmann, Karlsruher Institut für Technologie,
Institut für Regelungs- und Steuerungssysteme
Folien aus Kapitel 3 "Lineare Dynamische Systeme" vom WS 21/22
- [6] <https://blog.poscope.com/stepper-motor-driver/> (5.5.22)
- [7] <https://www.makerguides.com/wp-content/uploads/2019/10/TB6600-Manual.pdf> (5.5.22)
- [8] Jasper Tan, Consultant for Advantir Innovations
jasper.tan@motionsolution.net
- [9] https://www.alibaba.com/product-detail/Servo-Motor-50w-11kw-Ac-Controller_1600083418395.html?spm=a2700.galleryofferlist.normal_offer.d_title.fc861143UDrh7F&s=p
- [10] https://www.alibaba.com/product-detail/220V-1-3kw-High-torque-ac_60692359974.html
- [11] https://www.alibaba.com/product-detail/ac-servo-motor-4kw-3-8kw_60745365757.html?spm=a2700.wholesale.0.0.5867a101fn5HNs
- [12] <https://www.pimzos.com/en/trs600.html>
- [13] <https://www.7pcb.com/trace-width-calculator.php>
- [14] Lecture slides "Robotics 1: Introduction to robotics" WS 21/22
Prof. Tamim Asfour, Karlsruhe Institute of Technology
H2T, Institut für Hochperformante Humanoide Technologien
- [15] <https://learnopencv.com/convex-hull-using-opencv-in-python-and-c/>(21.8.22)

- [16] Lecture slides "Cognitive Systems" SS 20
Prof. Alex Waibel, Karlsruhe Institute of Technology
Interactive Systems Lab, IAR
- [17] <https://www.artstation.com/andreasmischok/blog>(1.5.22)
- [18] <https://shop.gcreate.com/products/nema-17-stepper-motor>(22.8.22)
- [19] <https://sg.element14.com/>(3.8.22)
- [20] <https://www.sparkfun.com/products/14321>(3.9.22)
- [21] <https://shopee.sg/>(3.8.22)
- [22] <https://3deshnik.ru/wiki/index.php/>(6.9.22)
- [23] <https://stackoverflow.com/questions/20677795/how-do-i-compute-the-intersection-point-of-two-lines>(18.09.22)

10 Appendix

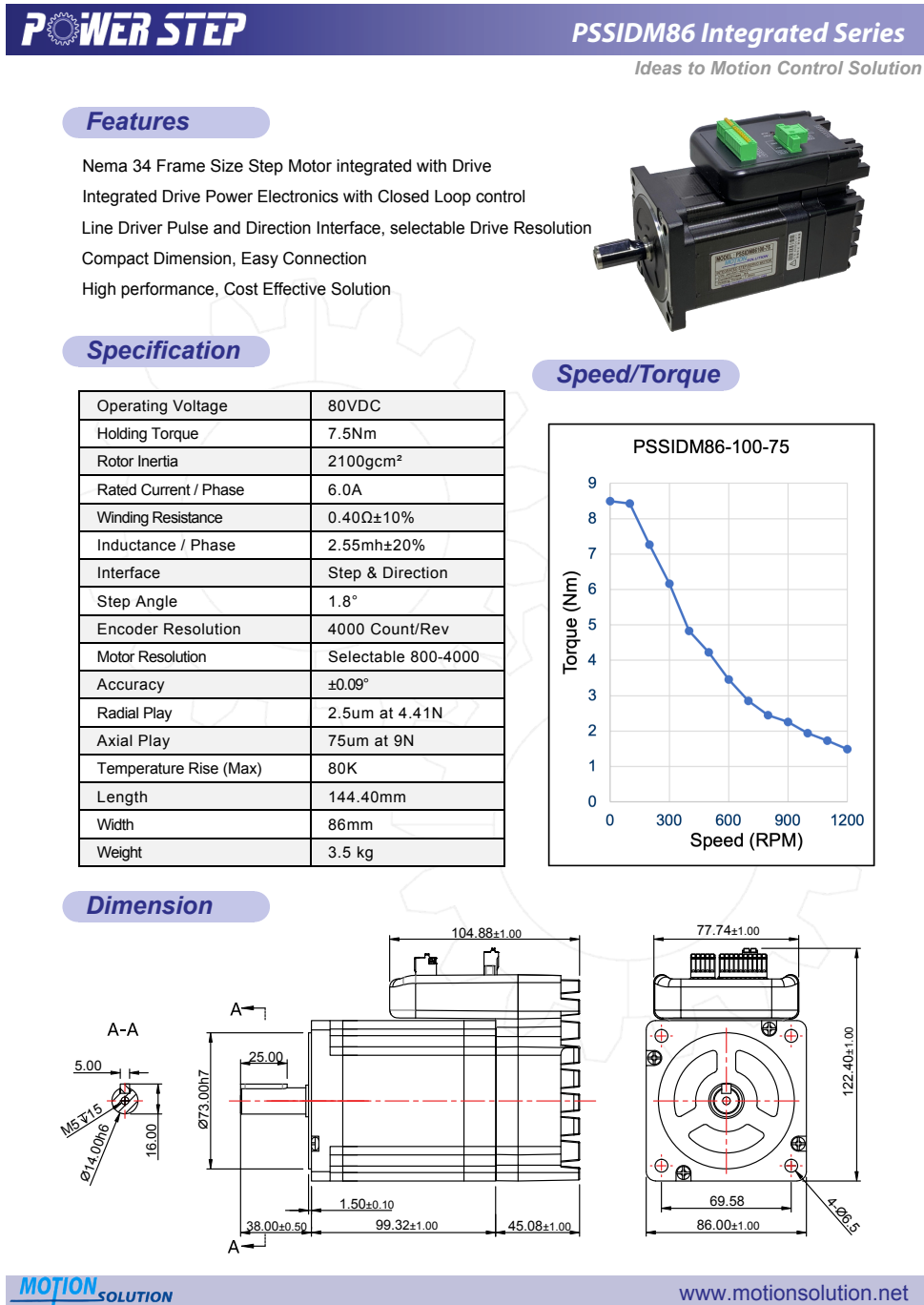


Figure 74: Data sheet of the currently used motor [8]